

What is an Image Edge:



Sculpture photograph by Henry Moore,



an artist's sketch of the sculpture

A few strokes, convey vital visual information:

Structure

Shading and so on ..

Edge: Rapid change in image intensity within small region

- Rapid change in image intensity are caused by various physical phenomena.

Sources of an Edge in Image:



two surfaces are made of the same material, if they have different surface orientations where they meet, they will likely receive different amounts of light from the light sources in the scene, and hence have different brightness values

Surface Normal Discontinuity

Depth Discontinuity

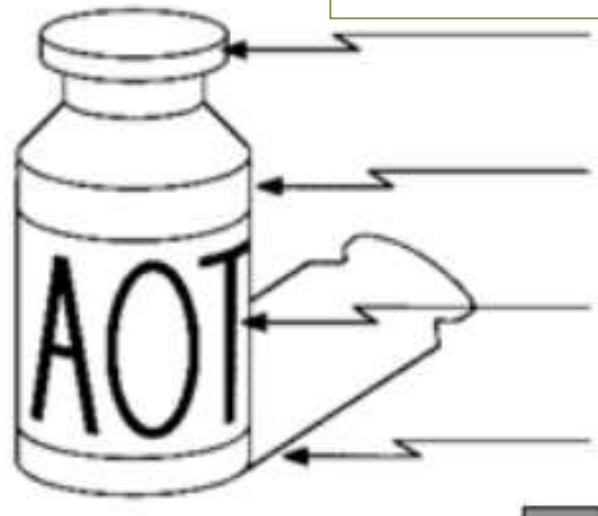
If one object is located in front of another, there will likely be a sudden change in intensity along the boundary between the two objects

Surface Reflectance Discontinuity

Illumination Discontinuity

if the surface is marked, such as the letters on the bottle here, it will have surface reflectance (material) discontinuities

bottle casts a sharp shadow on the background, resulting in a significant difference in the amount of light falling within and outside the shadow



Rapid change in image intensity are caused by various physical phenomena.

Types of Edges:



Step edge



Transition between two intensity levels occurring ideally over a distance of 1 pixel

Ramp edge



Transition between two intensity levels gradually over a distance of several pixels due to blurring

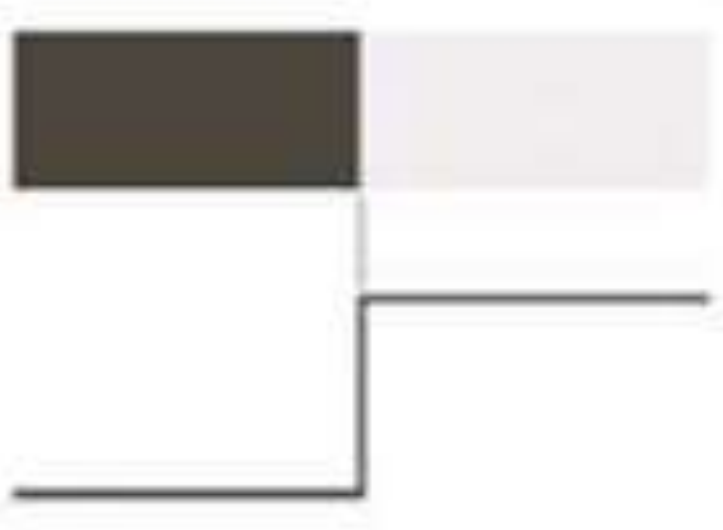
slope of the ramp is directly proportional to the degree of blurring

Roof edge

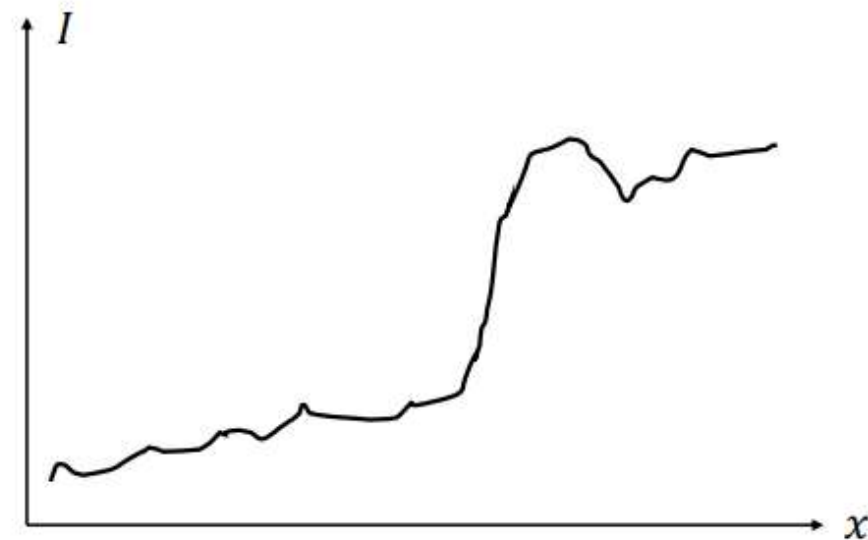


Intensity changes are not instantaneous but occur over a finite distance. The edge width determines the line thickness and sharpness

Noise Impact on Edge:



Step edge



Noise Impact on Edge

Properties and Performance Requirement: Edge Operator



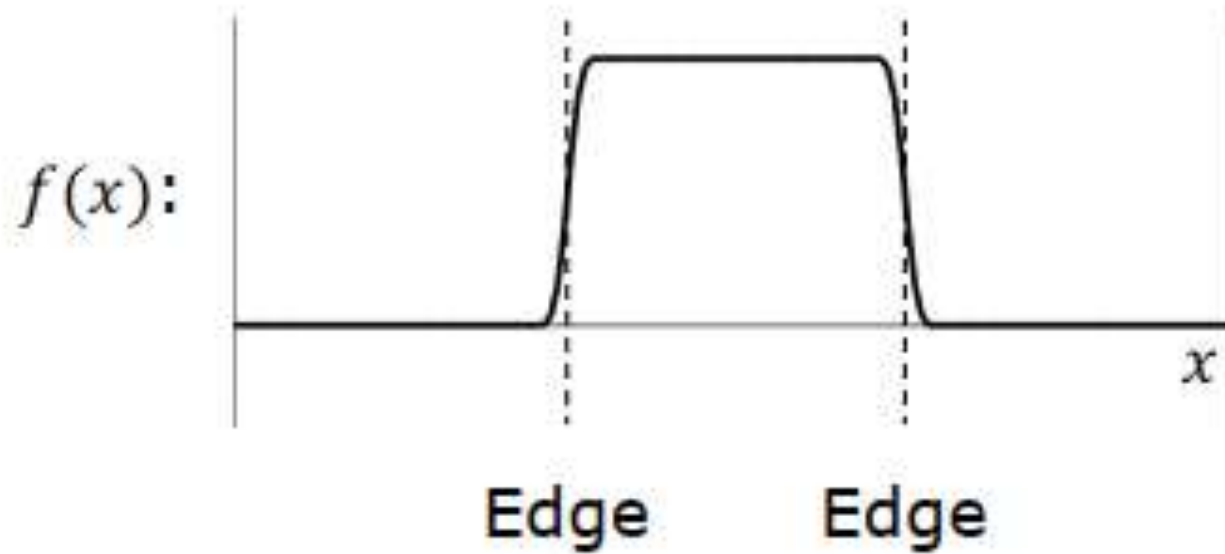
We want an Edge Operator that produces:

- Edge **Position**
- Edge **Magnitude** (Strength)
- Edge **Orientation** (Direction)

Performance Requirements:

- High **Detection Rate**
- Good **Localization**
- **Low Noise Sensitivity**

1D Edge detection:



2-Order:

$$\frac{\partial^2 f}{\partial x^2} = f(x+1) + f(x-1) - 2f(x)$$

Basic Calculus:

Derivative of a continuous function.

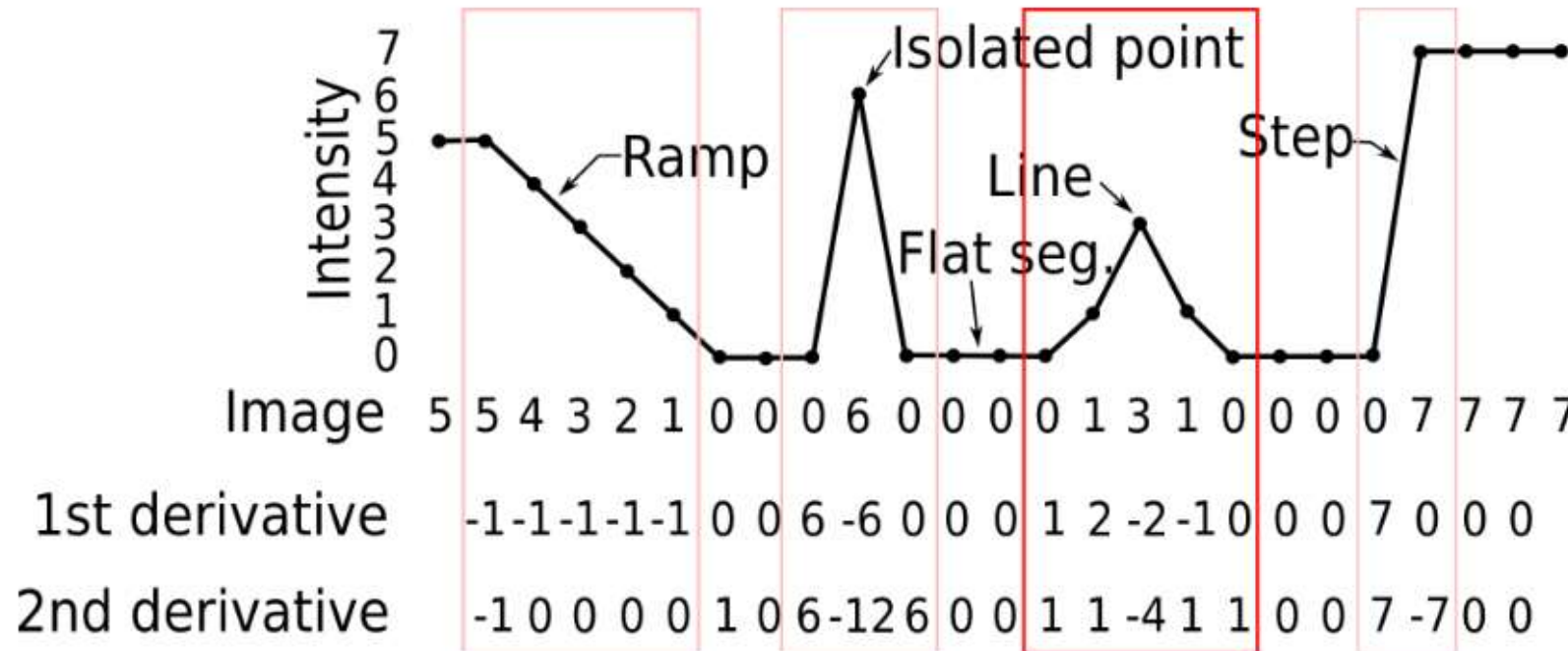
It represents the amount of change in the function.

One-Dimensional Signal $f(x)$

1-Order:

$$\frac{\partial f}{\partial x} = f(x+1) - f(x)$$

1D Edge detection:



One-Dimensional Signal $f(x)$

1-Order:

$$\frac{\partial f}{\partial x} = f(x+1) - f(x)$$

2-Order:

$$\frac{\partial^2 f}{\partial x^2} = f(x+1) + f(x-1) - 2f(x)$$

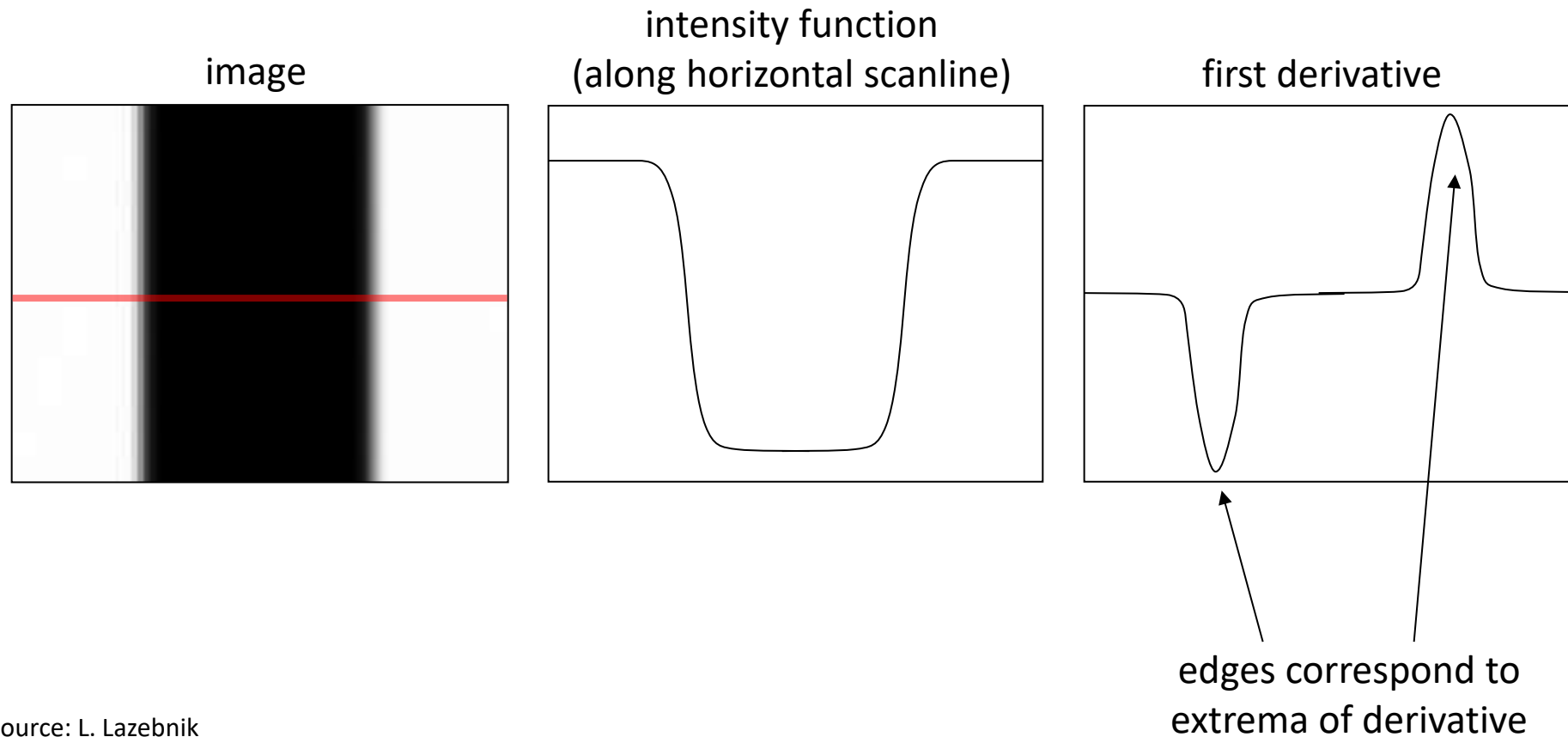
Formalization of the three types of finite differences:



		 $\epsilon = 1$	$\epsilon = 1$
(a) Forward	Finite diff.	$\frac{\partial I[x_m, y_n]}{\partial x} \approx \frac{I[x_m + \epsilon, y_n] - I[x_m, y_n]}{\epsilon}$	$\frac{\partial I[x_m, y_n]}{\partial y} \approx \frac{I[x_m, y_n + \epsilon] - I[x_m, y_n]}{\epsilon}$
	Kernel	$\begin{bmatrix} 0 & -1 & 1 \end{bmatrix}$	$\begin{bmatrix} 0 \\ -1 \\ 1 \end{bmatrix}$
(b) Backward	Finite diff.	$\frac{\partial I[x_m, y_n]}{\partial x} \approx \frac{I[x_m, y_n] - I[x_m - \epsilon, y_n]}{\epsilon}$	$\frac{\partial I[x_m, y_n]}{\partial y} \approx \frac{I[x_m, y_n] - I[x_m, y_n - \epsilon]}{\epsilon}$
	Kernel	$\begin{bmatrix} -1 & 1 & 0 \end{bmatrix}$	$\begin{bmatrix} -1 \\ 1 \\ 0 \end{bmatrix}$
(c) Central	Finite diff.	$\frac{\partial I[x_m, y_n]}{\partial x} \approx \frac{I[x_m + \epsilon, y_n] - I[x_m - \epsilon, y_n]}{\epsilon}$	$\frac{\partial I[x_m, y_n]}{\partial y} \approx \frac{I[x_m, y_n + \epsilon] - I[x_m, y_n - \epsilon]}{\epsilon}$
	Kernel	$\begin{bmatrix} -1 & 0 & 1 \end{bmatrix}$	$\begin{bmatrix} -1 \\ 0 \\ 1 \end{bmatrix}$

<https://towardsdatascience.com/image-derivative-8a07a4118550/>

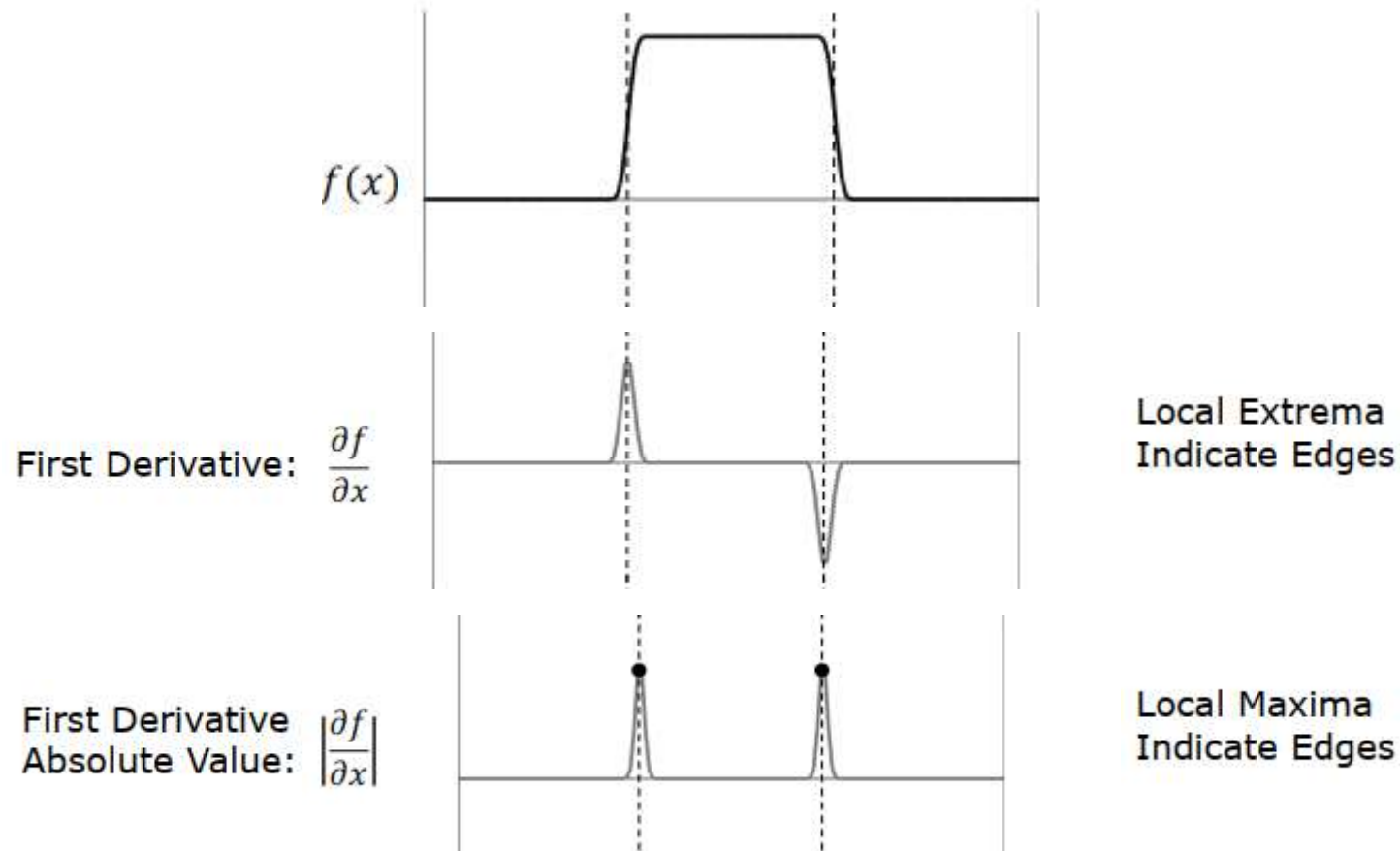
1D Edge Characterization::



Source: L. Lazebnik

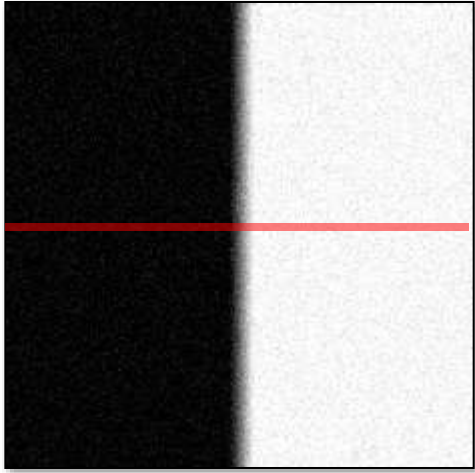
Edge detection using first derivative:

Edge is a rapid change in image intensity in a small region.



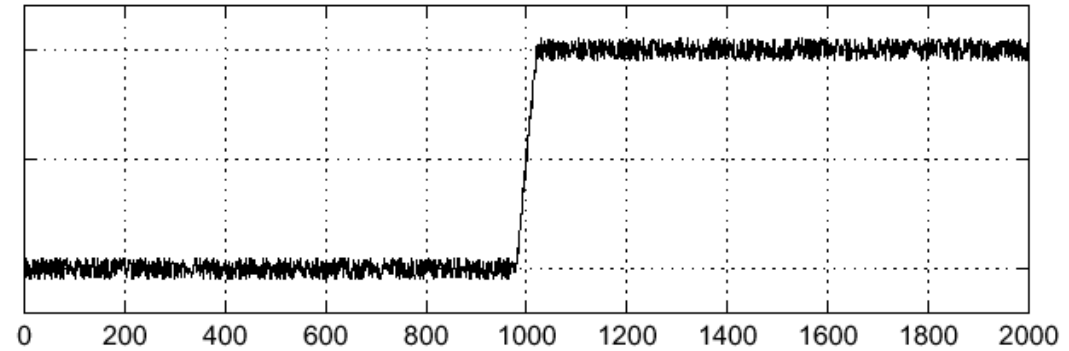
Provides Both Location and Strength of an Edge

Edge: Noise Impact

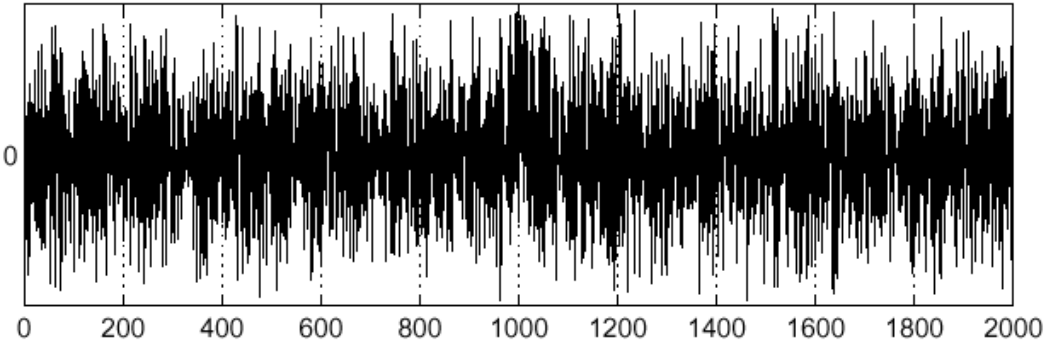


Noisy input image

$$f(x)$$

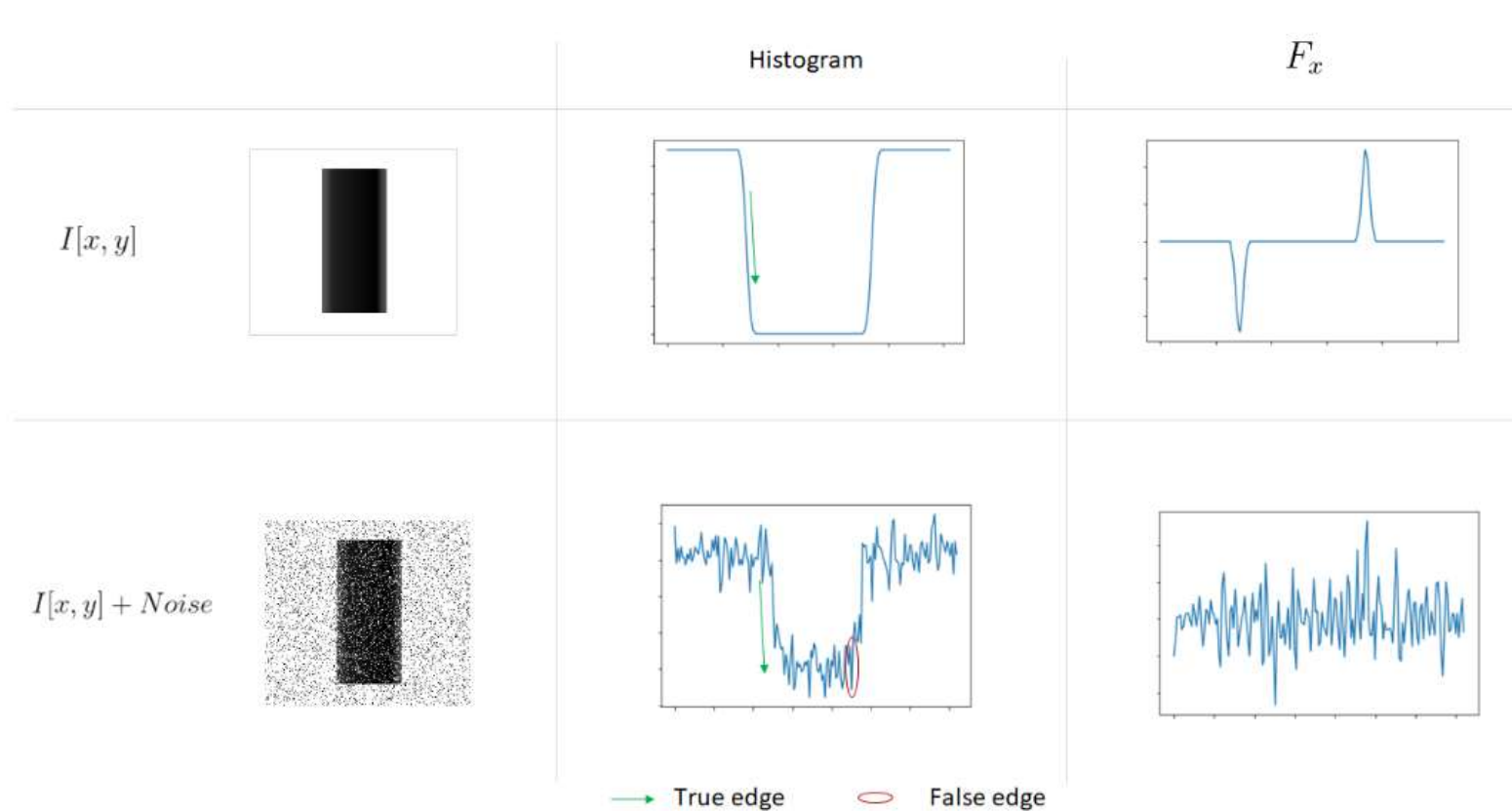


$$\frac{d}{dx}f(x)$$



Where is the edge?

Noise impact on Image:



Tab.2: Comparison of derivatives calculated on the same image $I[x, y]$ with and without the presence of salt-and-pepper noise (Source: Image by me)

One way to reduce *false edges* is to blur the image

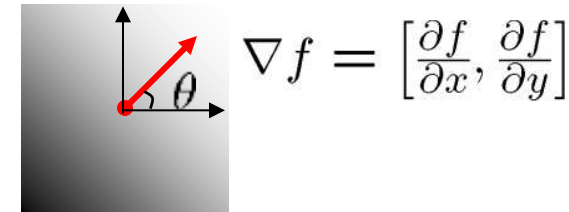
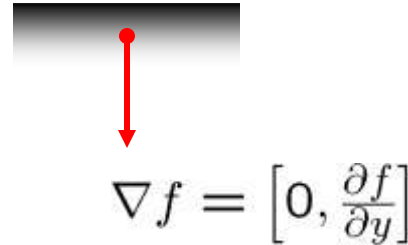
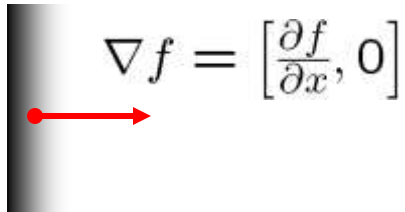
Image gradient

Gradient (Partial Derivatives) represents the direction of most rapid change in intensity

- The *gradient* of an image: $\nabla f = \left[\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y} \right]$



The gradient points in the direction of most rapid increase in intensity



The *edge strength* is given by the gradient magnitude:

$$\|\nabla f\| = \sqrt{\left(\frac{\partial f}{\partial x}\right)^2 + \left(\frac{\partial f}{\partial y}\right)^2}$$

The gradient direction is given by:

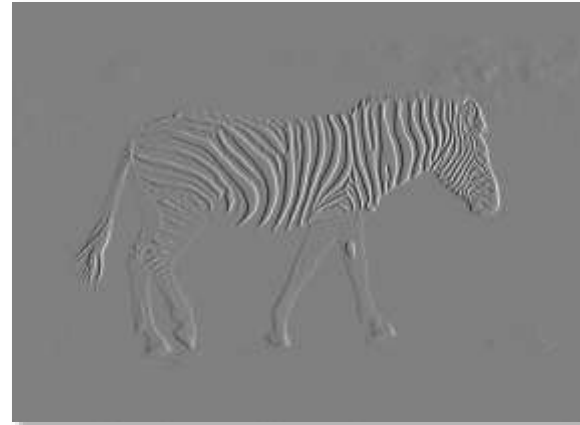
$$\theta = \tan^{-1} \left(\frac{\partial f}{\partial y} / \frac{\partial f}{\partial x} \right)$$

$\nabla^2 f$
↓
Laplacian

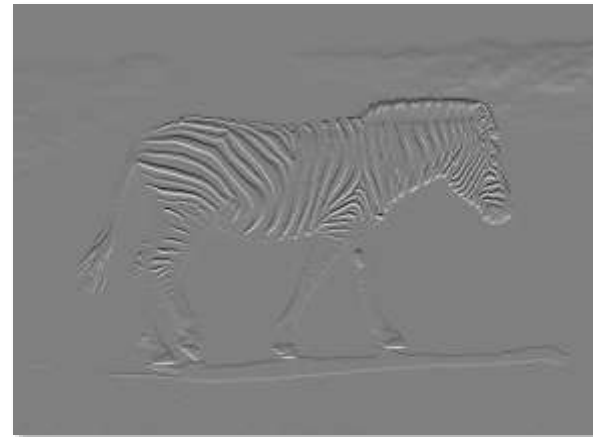
Gradient magnitude



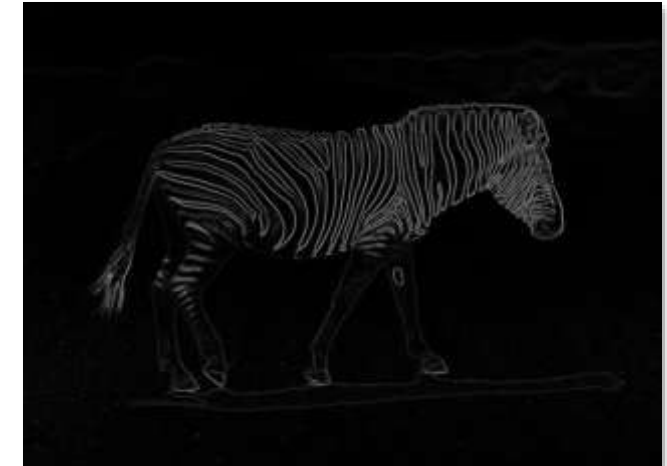
f



$\frac{\partial f}{\partial x}$



$\frac{\partial f}{\partial y}$



$$\|\nabla f\| = \sqrt{\left(\frac{\partial f}{\partial x}\right)^2 + \left(\frac{\partial f}{\partial y}\right)^2}$$

Blurring with moving average: Prewitt operator:



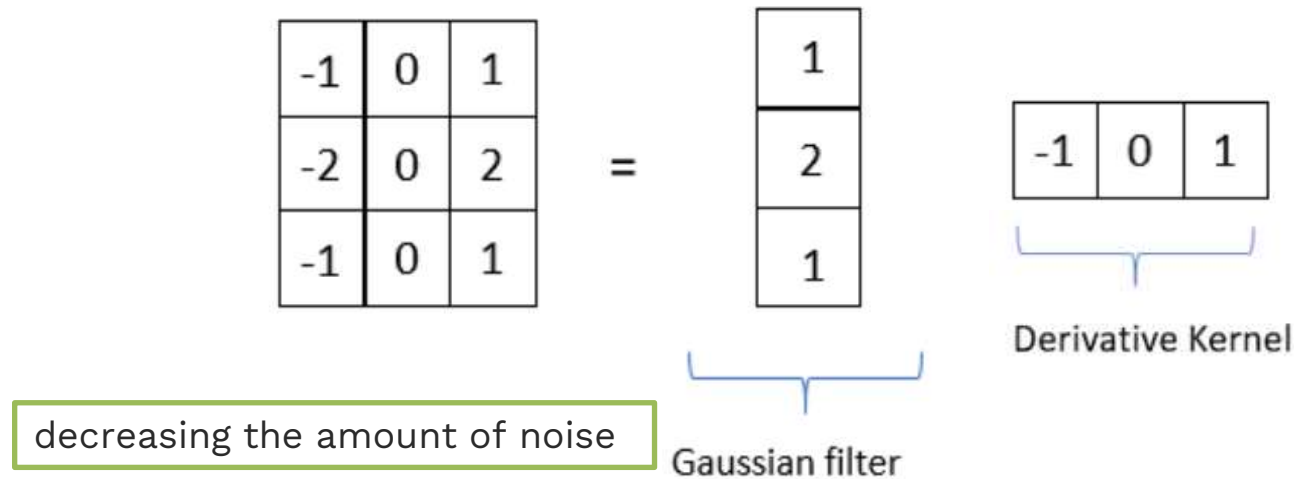
$$\frac{1}{3} \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix} = \frac{1}{3} \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} \begin{bmatrix} -1 & 0 & 1 \end{bmatrix}$$

Mean smoothing
Derivative Kernel

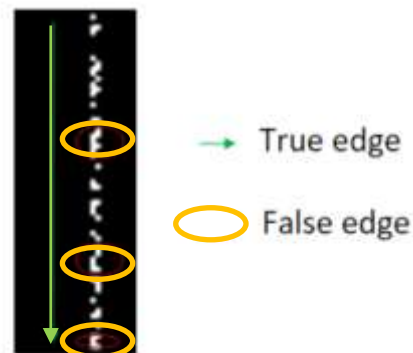
decreasing the amount of noise

symmetrical derivative Kernels of size 1x3 stacked on top of each other

Blurring with moving average: Sobel operator:



symmetrical derivative Kernels of size 1×3 stacked on top of each other

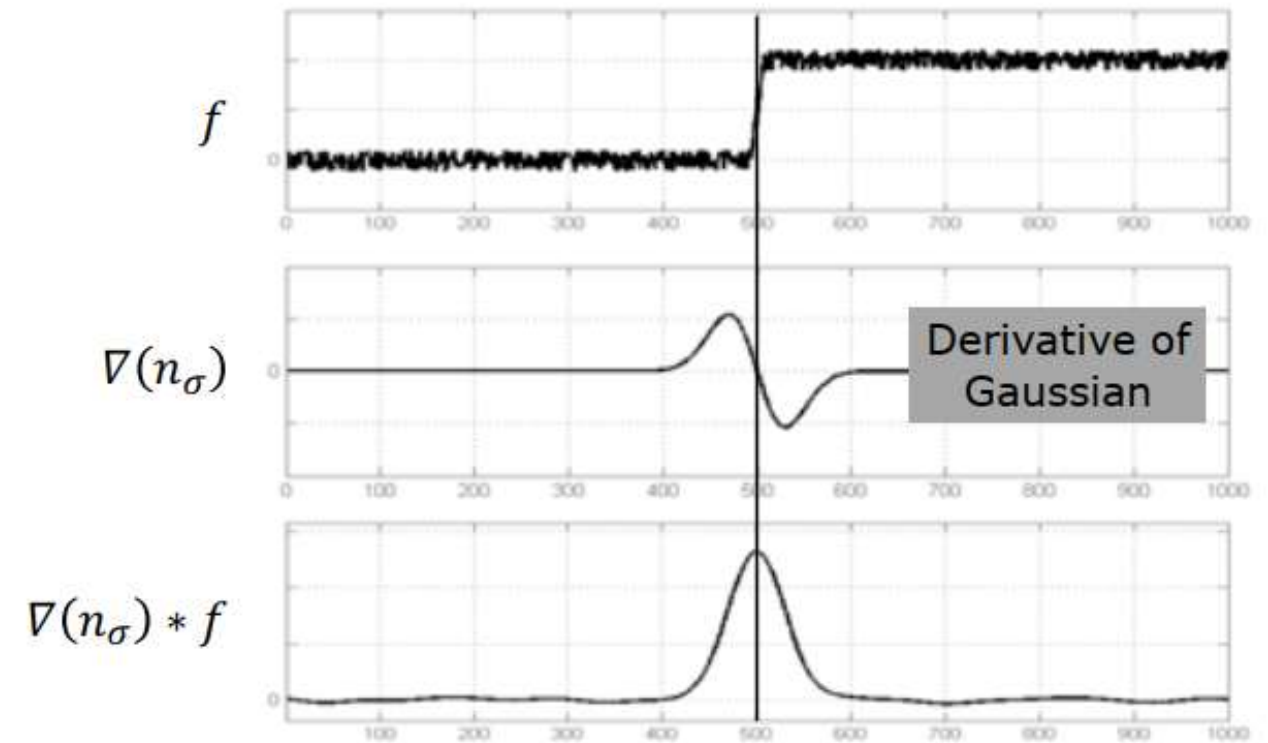
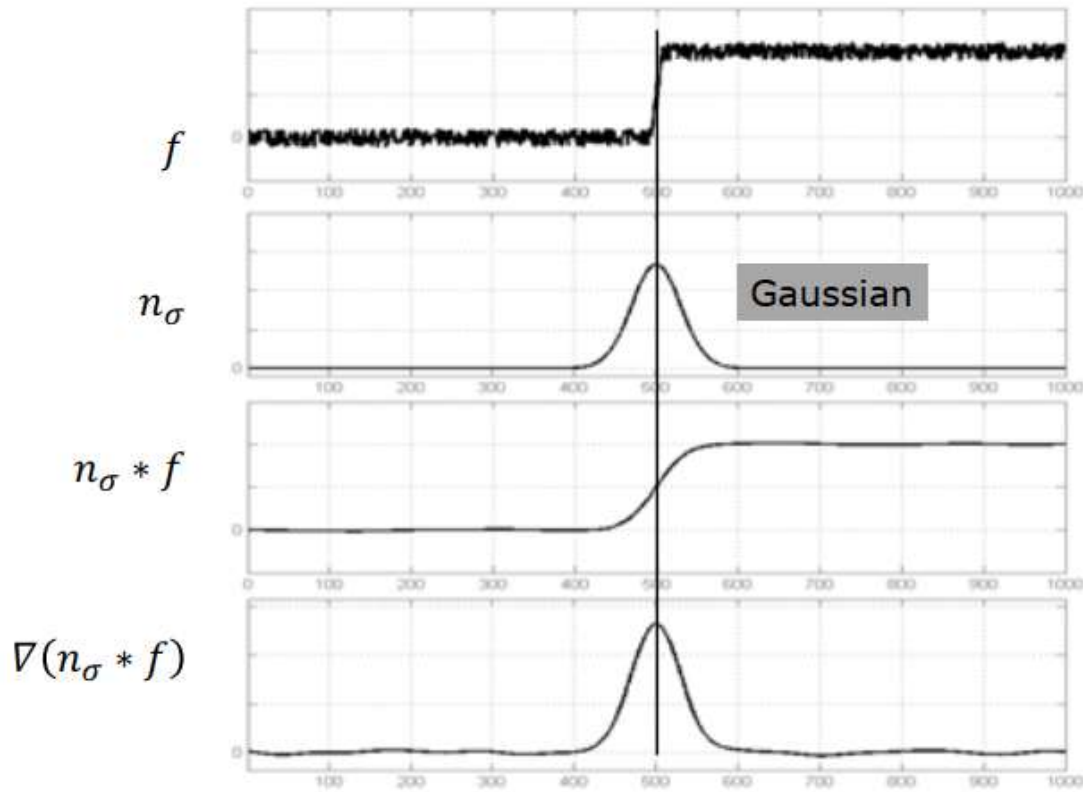


Blurring can reduce this phenomenon but not definitively eliminate it.

These effects can be reduced by using *Canny*, an edge detector that applies non-maximum suppression

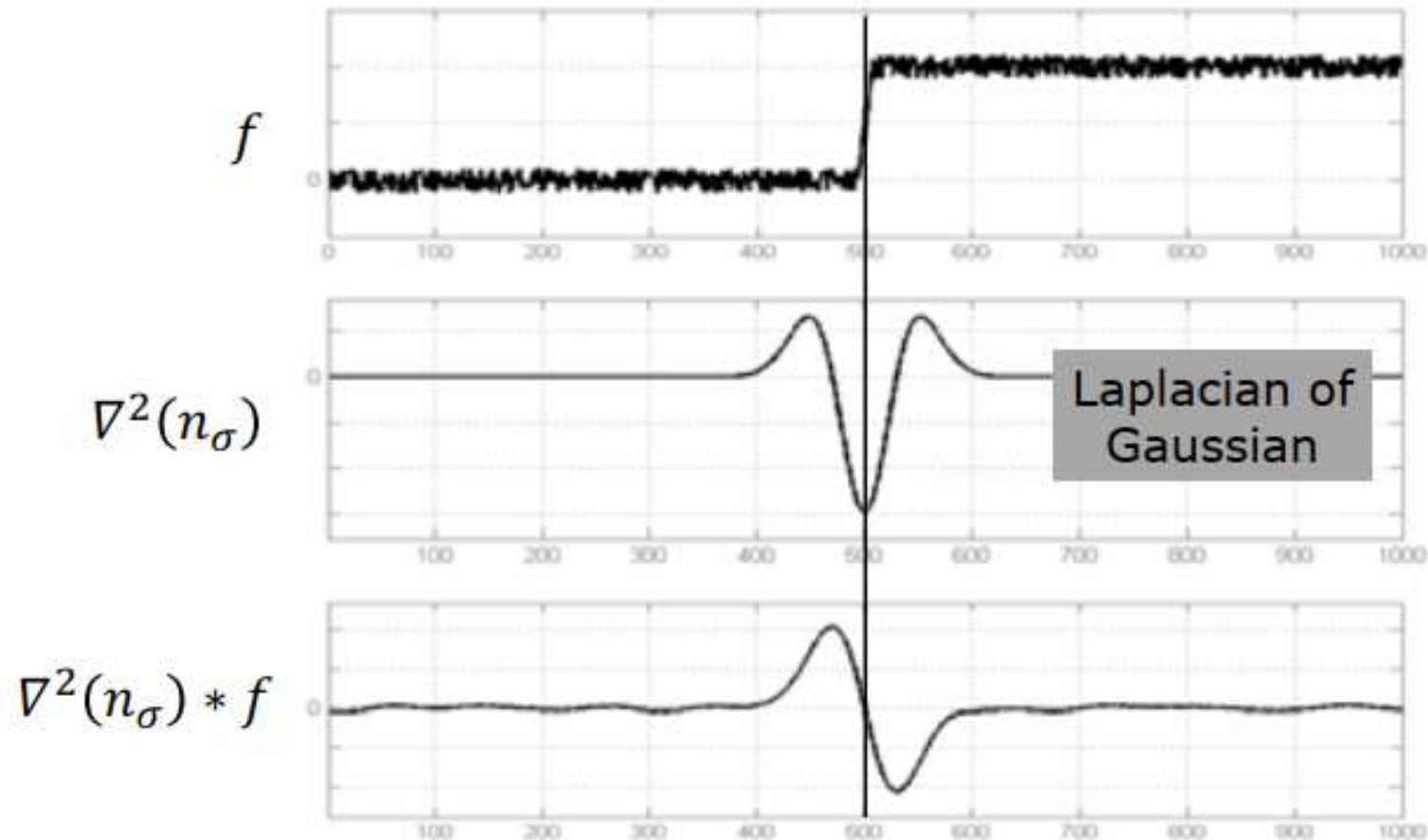
Gaussian smooth

$$\nabla(n_\sigma * f) = \nabla(n_\sigma) * f$$



Laplacian of Gaussian

$$\nabla^2(n_\sigma * f) = \nabla^2(n_\sigma) * f$$



The size of a discrete gradient operator affects localization by introducing a trade-off between noise sensitivity and localization accuracy

Discrete Gradient operator:

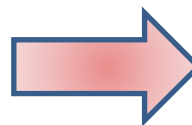
Finite difference approximations:



Gradient	Roberts	Prewitt	Sobel (3x3)	Sobel (5x5)
$\frac{\partial I}{\partial x}$	$\begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix}$	$\begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}$	$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$	$\begin{bmatrix} -1 & -2 & 0 & 2 & 1 \\ -2 & -3 & 0 & 3 & 2 \\ -3 & -5 & 0 & 5 & 3 \\ -2 & -3 & 0 & 3 & 2 \\ -1 & -2 & 0 & 2 & 1 \\ 1 & 2 & 3 & 2 & 1 \\ 2 & 3 & 5 & 3 & 2 \\ 0 & 0 & 0 & 0 & 0 \\ -2 & -3 & -5 & -3 & -2 \\ -1 & -2 & -3 & -2 & -1 \end{bmatrix}$
$\frac{\partial I}{\partial y}$	$\begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$	$\begin{bmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ -1 & -1 & -1 \end{bmatrix}$	$\begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}$	

Increase localization accuracy,

Increase sensitivity to noise



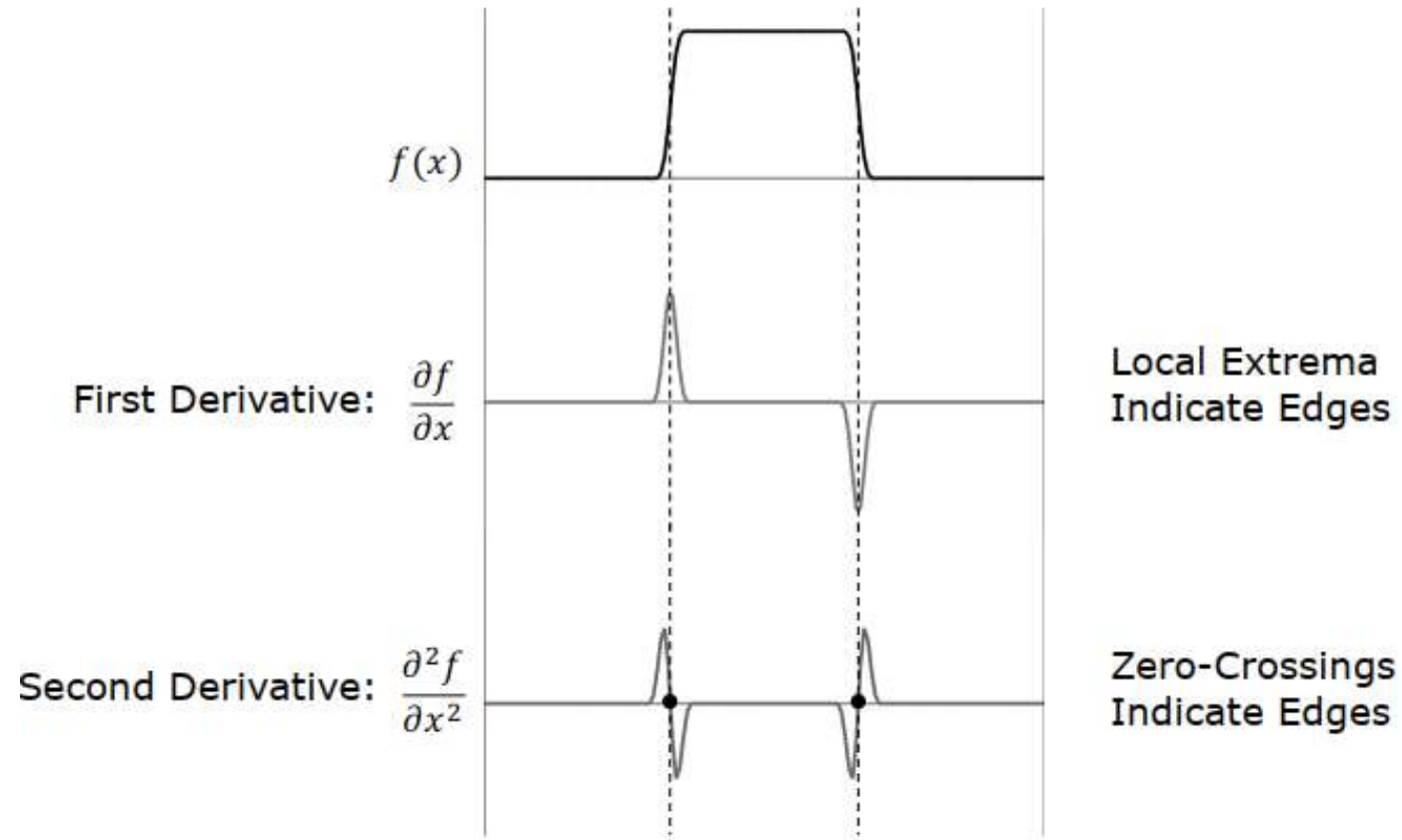
Decreases the localization accuracy,

Decreases sensitivity to noise

Gradient using Sobel operator:



Edge detection using Laplacian (second derivatives):



Laplacian as Edge detector:



Laplacian: Sum of Second Derivatives

$$\nabla^2 I = \frac{\partial^2 I}{\partial x^2} + \frac{\partial^2 I}{\partial y^2}$$

- Edges are “zero-crossings” in Laplacian of image
- Laplacian does not provide directions of edges

Discrete Laplacian operator:



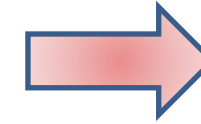
$I_{i-1,j+1}$	$I_{i,j+1}$	$I_{i+1,j+1}$
$I_{i-1,j}$	$I_{i,j}$	$I_{i+1,j}$
$I_{i-1,j-1}$	$I_{i,j-1}$	$I_{i+1,j-1}$

ε

Finite difference approximations:

$$\frac{\partial^2 I}{\partial x^2} \approx \frac{1}{\varepsilon^2} (I_{i-1,j} - 2I_{i,j} + I_{i+1,j})$$

$$\frac{\partial^2 I}{\partial y^2} \approx \frac{1}{\varepsilon^2} (I_{i,j-1} - 2I_{i,j} + I_{i,j+1})$$



$$\nabla^2 I = \frac{\partial^2 I}{\partial x^2} + \frac{\partial^2 I}{\partial y^2}$$

Convolution Mask:

$$\nabla^2 \approx \frac{1}{\varepsilon^2} \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

OR

$$\nabla^2 \approx \frac{1}{6\varepsilon^2} \begin{bmatrix} 1 & 4 & 1 \\ 4 & -20 & 4 \\ 1 & 4 & 1 \end{bmatrix} \quad \text{(More Accurate)}$$

Laplacian edge detector:



Image (I)



Laplacian
(0 maps to 128)



Laplacian
"Zero Crossings"

Since an image cannot be displayed with negative values, we use the level 128 for 0, such that pixels darker than 128 are negative and pixels brighter than 128 are positive

Wherever there is a zero-crossing, the pixel value will be exactly 128, with large positive and negative values around it.

Differences between Gradient and Laplacian operators:



Gradient	vs.	Laplacian
Provides location, magnitude and direction of the edge.		Provides only location of the edge.
Detection using Maxima Thresholding.		Detection based on Zero-Crossing.
Non-linear operation. Requires two convolutions.		Linear Operation. Requires only one convolution.

An operator that has the best of both?

Canny edge detector:

The **Canny edge detector** is an [edge detection](#) operator that uses a multi-stage [algorithm](#) to detect a wide range of edges in images. It was developed by [John F. Canny](#) in 1986.

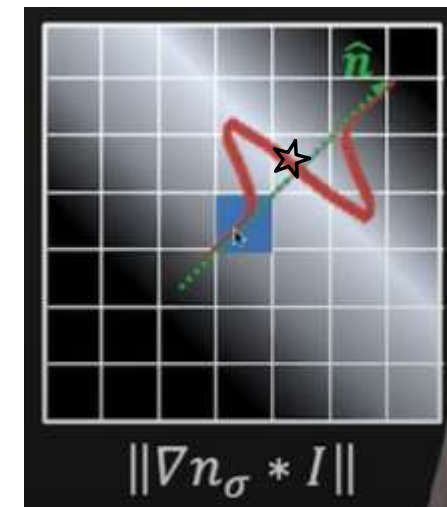
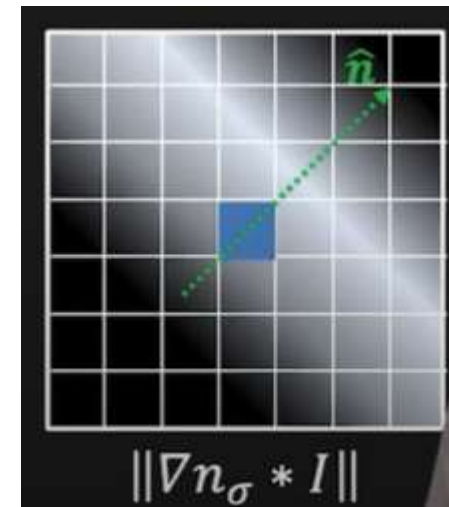
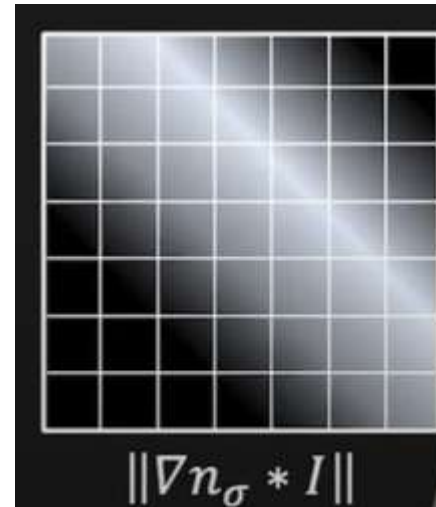


- Smooth Image with 2D Gaussian: $n_\sigma * I$
- Compute Image Gradient using Sobel Operator: $\nabla n_\sigma * I$
- Find Gradient Magnitude at each pixel: $\|\nabla n_\sigma * I\|$
- Find Gradient Orientation at each Pixel:
- Find Zero Crossings in Laplacian to find the edge location

$$\hat{n} = \frac{\nabla n_\sigma * I}{\|\nabla n_\sigma * I\|}$$

- Compute Laplacian along the Gradient Direction $\hat{n}$ at each pixel

$$\frac{\partial^2(n_\sigma * I)}{\partial \hat{n}^2}$$



Canny edge detection algorithm:

It was developed by John F. Canny in 1986.
Canny operator is a multi-stage algorithm that detects wide range of edges.



Stages

- 1.Smoothing for noise removal.
- 2.Finding Gradients.
- 3.None-maximum suppression.
- 4.Double Thresholding.
- 5.Edge Tracking by hysteresis.

Noise reduction:



The equation for a Gaussian filter kernel of size $(2k+1) \times (2k+1)$ is given by:

$$H_{ij} = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{(i - (k+1))^2 + (j - (k+1))^2}{2\sigma^2}\right); 1 \leq i, j \leq (2k+1)$$

```
1  import numpy as np
2
3  def gaussian_kernel(size, sigma=1):
4      size = int(size) // 2
5      x, y = np.mgrid[-size:size+1, -size:size+1]
6      normal = 1 / (2.0 * np.pi * sigma**2)
7      g = np.exp(-((x**2 + y**2) / (2.0*sigma**2))) * normal
8      return g
```

Gradient Detection:

Edges correspond to a change of pixels' intensity.



To detect it, the easiest way is to apply filters that highlight this intensity change in both directions: horizontal (x) and vertical (y)

It can be implemented by convolving I with Sobel kernels

$$K_x = \begin{pmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{pmatrix}, K_y = \begin{pmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{pmatrix}$$

Sobel filters for both direction (horizontal and vertical)

$$|G| = \sqrt{I_x^2 + I_y^2},$$

$$\theta(x, y) = \arctan\left(\frac{I_y}{I_x}\right)$$

```

1  from scipy import ndimage
2
3  def sobel_filters(img):
4      Kx = np.array([[ -1,  0,  1], [-2,  0,  2], [-1,  0,  1]], np.float32)
5      Ky = np.array([[ 1,  2,  1], [0,  0,  0], [-1, -2, -1]], np.float32)
6
7      Ix = ndimage.filters.convolve(img, Kx)
8      Iy = ndimage.filters.convolve(img, Ky)
9
10     G = np.hypot(Ix, Iy)
11     G = G / G.max() * 255
12     theta = np.arctan2(Iy, Ix)
13
14     return (G, theta)

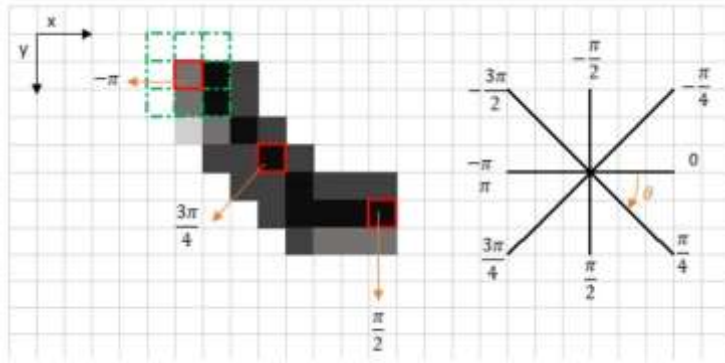
```

Non-Maximum Suppression:



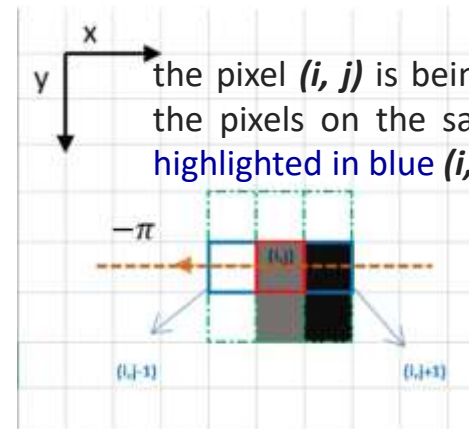
The final image should have thin edges.

the algorithm goes through all the points on the gradient intensity matrix and finds the pixels with the maximum value in the edge directions.



The upper left corner red box present on the above image, represents an intensity pixel of the Gradient Intensity matrix being processed.

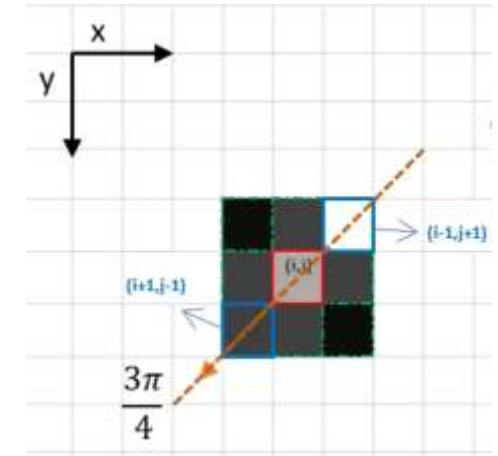
The corresponding edge direction is represented by the orange arrow with an angle of $-\pi$ radians (± 180 degrees).



The more intense one is kept. Pixel $(i, j-1)$ seems to be more intense, because it is white (value of 255).

Hence, the intensity value of the current pixel (i, j) is set to 0

If there are no pixels in the edge direction having more intense values, then the value of the current pixel is kept.



Non-Maximum Suppression: Algorithm



```

1  def non_max_suppression(img, D):
2      M, N = img.shape
3      Z = np.zeros((M,N), dtype=np.int32)
4      angle = D * 180. / np.pi
5      angle[angle < 0] += 180
6
7
8      for i in range(1,M-1):
9          for j in range(1,N-1):
10             try:
11                 q = 255
12                 r = 255
13
14                 #angle 0
15                 if (0 <= angle[i,j] < 22.5) or (157.5 <= angle[i,j] <= 180):
16                     q = img[i, j+1]
17                     r = img[i, j-1]
18
19                 #angle 45
20                 elif (22.5 <= angle[i,j] < 67.5):
21                     q = img[i+1, j-1]
22                     r = img[i-1, j+1]
23
24                 #angle 90
25                 elif (67.5 <= angle[i,j] < 112.5):

```

```

26             #angle 135
27             elif (112.5 <= angle[i,j] < 157.5):
28                 q = img[i-1, j-1]
29                 r = img[i+1, j+1]
30
31             if (img[i,j] >= q) and (img[i,j] >= r):
32                 Z[i,j] = img[i,j]
33             else:
34                 Z[i,j] = 0
35
36             except IndexError as e:
37                 pass
38
39         return Z

```



Result of the non-max suppression.

Double Threshold



The double threshold step aims at identifying 3 kinds of pixels:
strong, weak, and non-relevant:

Strong pixels:

The pixels that have an intensity so high that we are sure they contribute to the final edge.

Weak pixels:

The pixels that have an intensity value that is not enough to be considered as strong ones, but not small enough to be considered as non-relevant for the edge detection.

Non-Relevant Pixels

Other pixels are considered as non-relevant for the edge.

All pixels having intensity between both thresholds are flagged as weak

High threshold

It is used to identify the strong pixels (intensity higher than the high threshold)

Low threshold

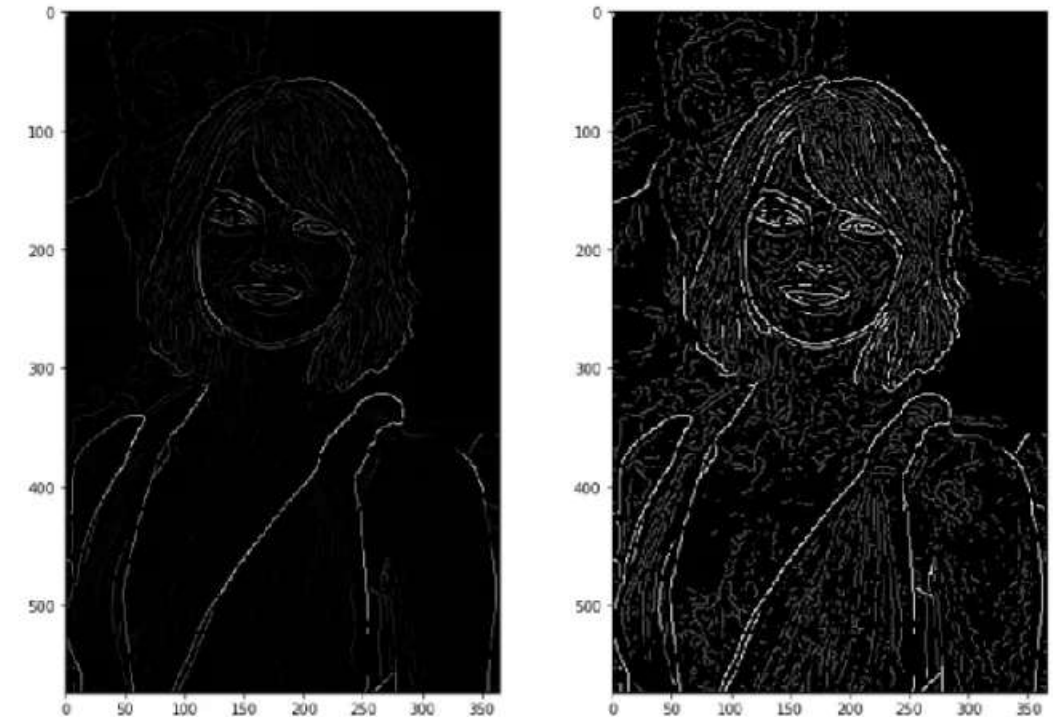
It is used to identify the non-relevant pixels (intensity lower than the low threshold)

Double Threshold program

```

1  def threshold(img, lowThresholdRatio=0.05, highThresholdRatio=0.09):
2
3      highThreshold = img.max() * highThresholdRatio;
4      lowThreshold = highThreshold * lowThresholdRatio;
5
6      M, N = img.shape
7      res = np.zeros((M,N), dtype=np.int32)
8
9      weak = np.int32(25)
10     strong = np.int32(255)
11
12     strong_i, strong_j = np.where(img >= highThreshold)
13     zeros_i, zeros_j = np.where(img < lowThreshold)
14
15     weak_i, weak_j = np.where((img <= highThreshold) & (img >= lowThreshold))
16
17     res[strong_i, strong_j] = strong
18     res[weak_i, weak_j] = weak
19
20     return (res, weak, strong)

```

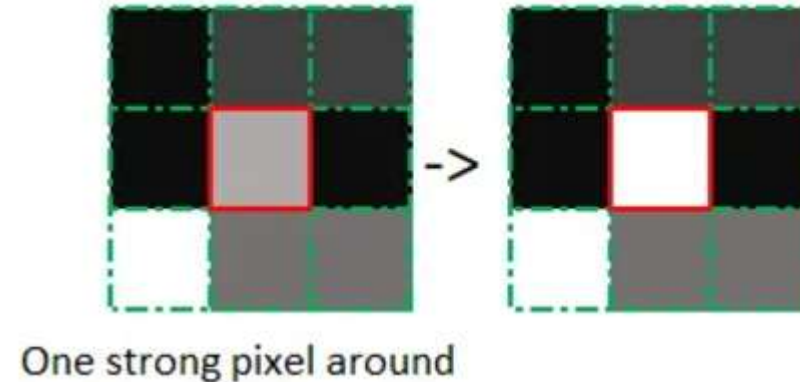
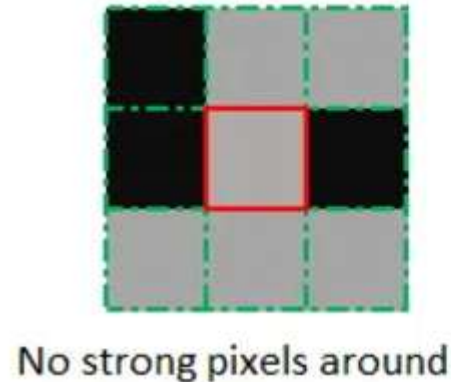


Non-Max Suppression image (left) — Threshold result (right); weak pixels in gray and strong ones in white.

<https://medium.com/data-science/canny-edge-detection-step-by-step-in-python-computer-vision-b49c3a2d8123>

Edge Tracking by Hysteresis:

the hysteresis consists of transforming weak pixels into strong ones, if and only if at least one of the pixels around the one being processed is a strong one, as described below:



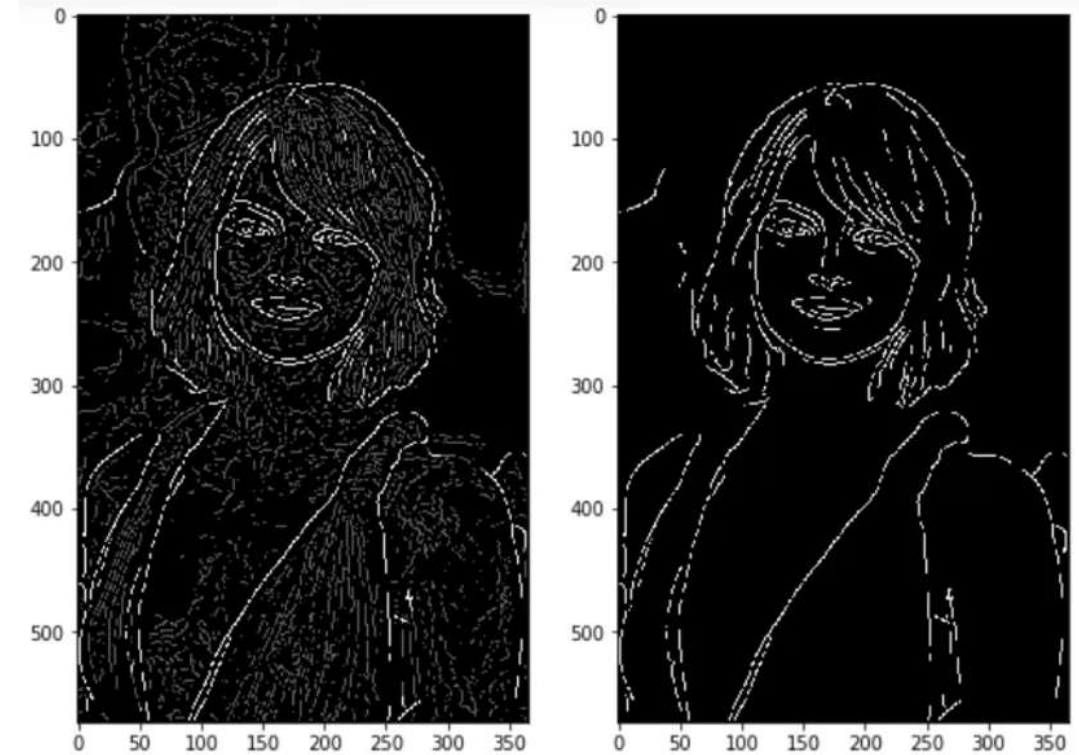
Edge Tracking by Hysteresis:



```

1  def hysteresis(img, weak, strong=255):
2      M, N = img.shape
3      for i in range(1, M-1):
4          for j in range(1, N-1):
5              if (img[i,j] == weak):
6                  try:
7                      if ((img[i+1, j-1] == strong) or (img[i+1, j] == strong) or (img[i+1, j+1] ==
8                          or (img[i, j-1] == strong) or (img[i, j+1] == strong)
9                          or (img[i-1, j-1] == strong) or (img[i-1, j] == strong) or (img[i-1, j+1]
10                             img[i, j] = strong
11                  except:
12                      img[i, j] = 0
13              except IndexError as e:
14                  pass
15      return img

```



Results of hysteresis process

Template matching

Template Matching is commonly used for object detection, *image recognition*, and *pattern recognition* tasks



Template matching refers to the image processing where we find similar templates in a source image by giving a base template.

The process of template matching is done by comparing each of the pixel values of the source image one at a time to the template image.

The output would be an array of similarity values when compared to the template image.

We can find the peaks in the resulting array of the template matching.



Template

Load a sample image:

```
import numpy as np
from skimage.io import imshow, imread
from skimage.color import rgb2gray
import matplotlib.pyplot as plt

sample = imread('flowers2.png')
sample_g = rgb2gray(sample)

fig, ax = plt.subplots(1,2,figsize=(10,5))
ax[0].imshow(sample)
ax[1].imshow(sample_g,cmap='gray')
ax[0].set_title('Colored Image',fontsize=15)
ax[1].set_title('Grayscale Image',fontsize=15)
plt.show()
```

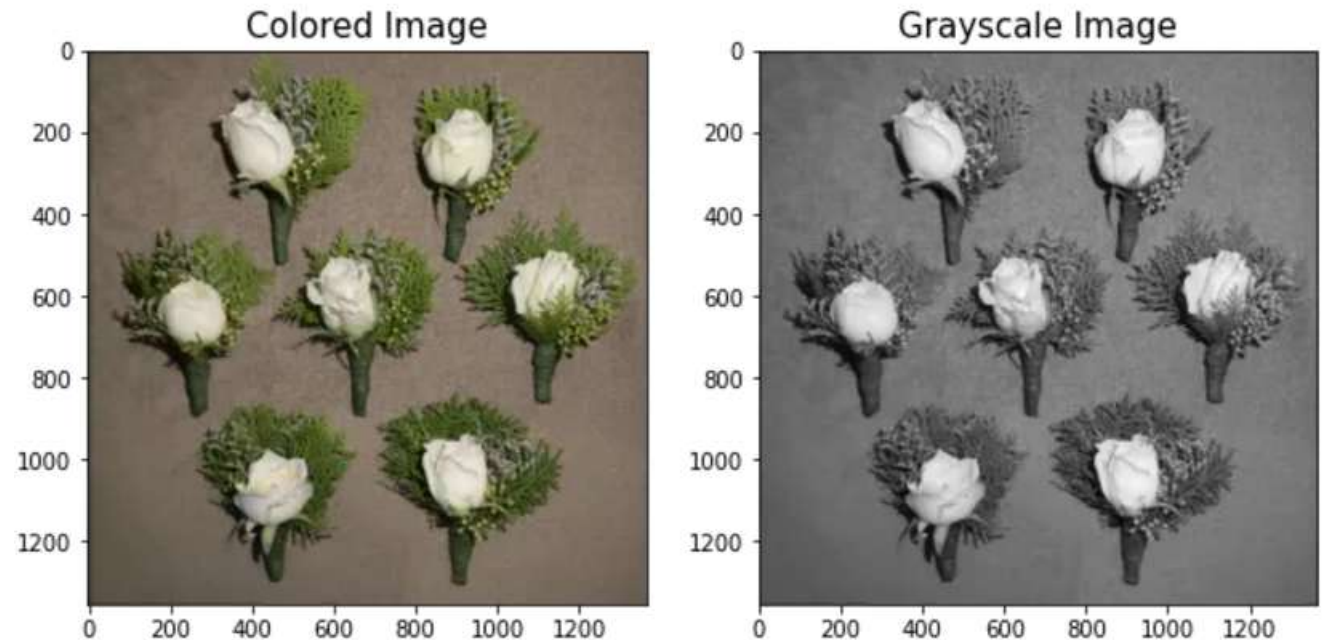


Figure 1: Sample Image (Image by Author)



Template selection



```
from matplotlib.patches import Rectangle

fig, ax = plt.subplots(1,2,figsize=(10,10))
ax[0].imshow(sample_g,cmap='gray')
ax[0].add_patch(Rectangle((800, 950), 200, 200, edgecolor='r',
facecolor='none'));
ax[0].set_title('Patch Location',fontsize=15)

#Showing Patch
patch = sample_g[950:1150, 800:1000]
ax[1].imshow(patch,cmap='gray')
ax[1].set_title('Patch',fontsize=15)
plt.show()
```

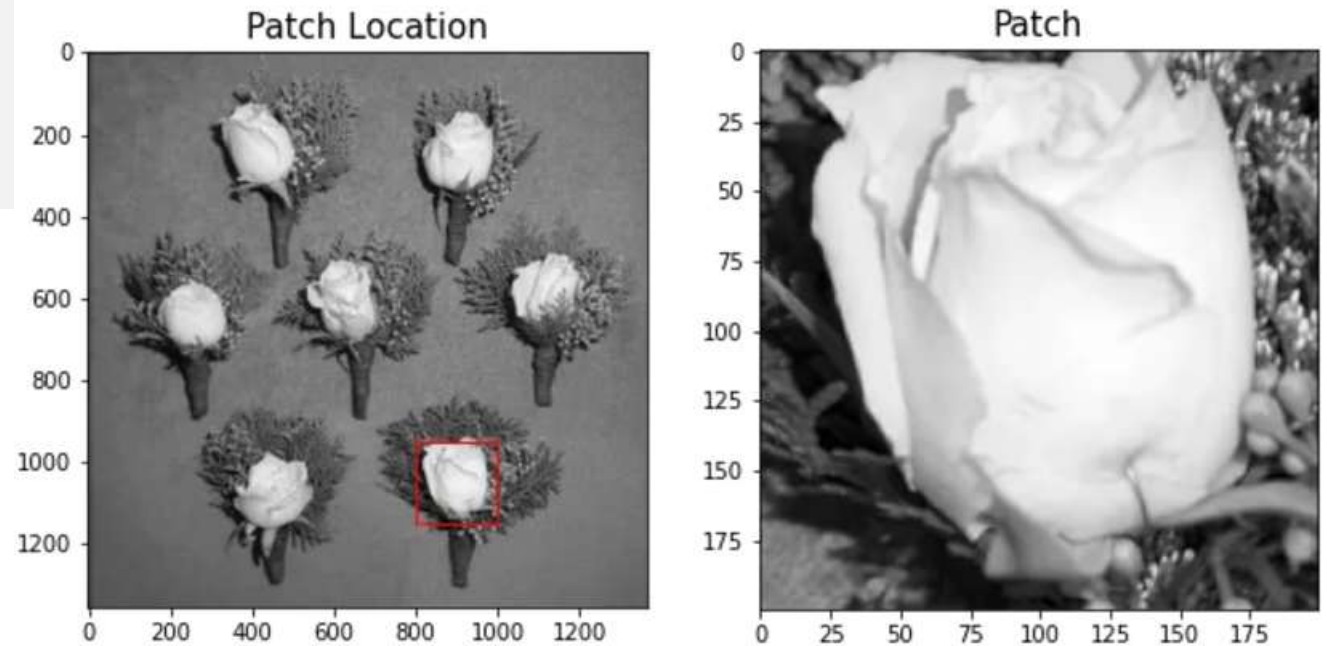


Figure 2: Template/Patch Image (Image by Author)

Template matching algorithm:



```
from skimage.feature import match_template
sample_mt = match_template(sample_g, patch)

fig, ax = plt.subplots(1,2,figsize=(10,10))
ax[0].imshow(sample_g,cmap='gray')
ax[1].imshow(sample_mt,cmap='gray')
ax[0].set_title('Grayscale',fontsize=15)
ax[1].set_title('Template Matching',fontsize=15);
```

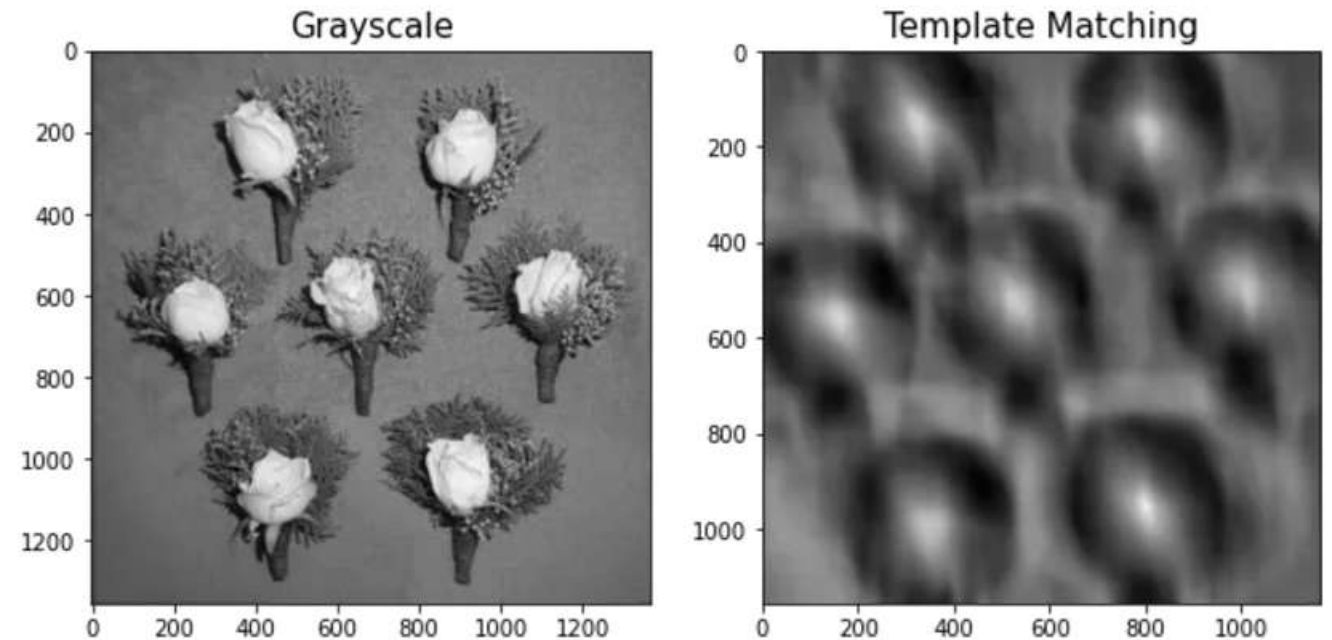


Figure 3: Performing Template Matching (Image by Author)

Feature Descriptor:

It is a simplified representation of the image that contains only the most important information about the image.



A lot of information, like the shape of the object, its color, the edges, background, etc



Much less information (only the shape and the edges) but it is still enough to differentiate the two images.

Feature Extraction:

This process transforms raw image data into numerical features that can be processed while preserving the essential information.

Feature extraction is a critical step in image processing and computer vision, involving the identification and representation of distinctive structures within an image.

These features are vital for various downstream tasks such as object detection, classification, and image matching.

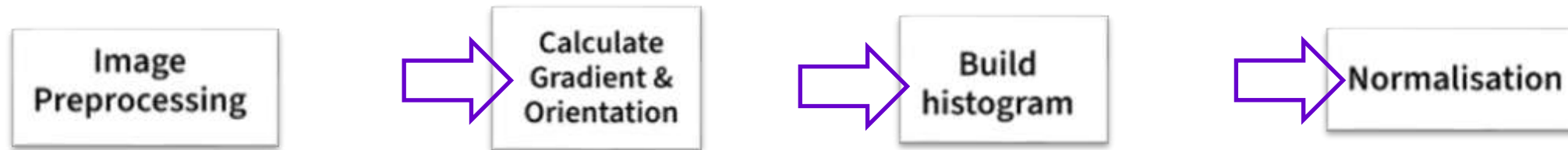
Histogram of Gradient (HoG)

HoG is a feature descriptor used in computer vision for object detection.

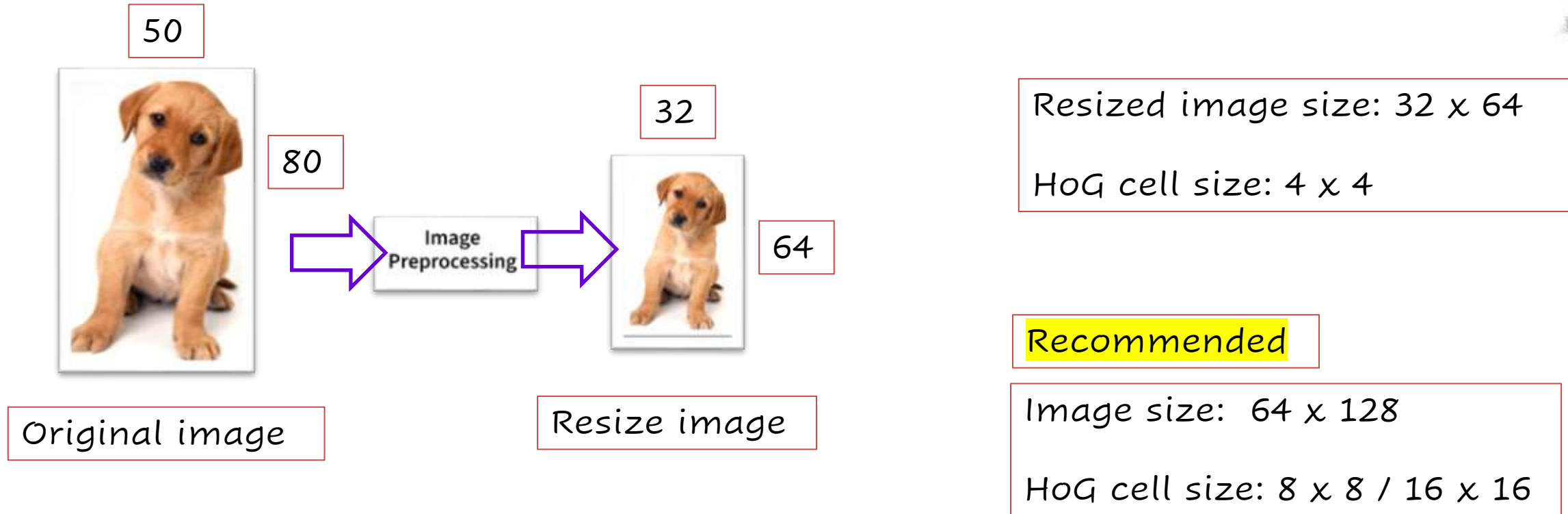


It captures the structure or the shape of an object by analyzing the distribution (histograms) of gradient orientations in localized portions of an image.

HoG is especially popular for human and vehicle detection.



HoG



resizing it to a standardized size of 32 pixels in width and 64 pixels in height.

This ensures consistency across all input images, making further analysis and feature extraction more reliable.

For a given cell (e.g., a 4x4 pixel block), calculate the gradient magnitude and orientation

P32 | C8

P64
C16

Resize image

Gradient computation (x dir.)

Gradient computation (y-dir)

Compute the gradient
magnitude and gradient
orientation at a pixel

Repeat for all pixels

C1 (4 x 4 cell)

121	10	78	96
48	152	64	125
145	78	85	89
154	214	56	200

Calculate
Gradient &
Orientation

$$\text{Magnitude} = \sqrt{G_x^2 + G_y^2}$$

$$\text{Orientation} = \arctan2(G_y, G_x)$$

NOTE: For edge pixels, use padding techniques



C1 (4 x 4 cell)

121	10	78	96
48	152	64	125
145	78	89	89
154	214	56	200

Calculate
Gradient &
Orientation

$$\text{Magnitude} = \sqrt{G_x^2 + G_y^2}$$

$$\text{Orientation} = \arctan2(G_y, G_x)$$

$$G(x) = 89 - 78 = 11$$

$$G(y) = 64 - 56 = 8$$

$$\text{Magnitude} = 13.6$$

$$\text{Orientation} = 36^\circ$$

HoG

Build histogram

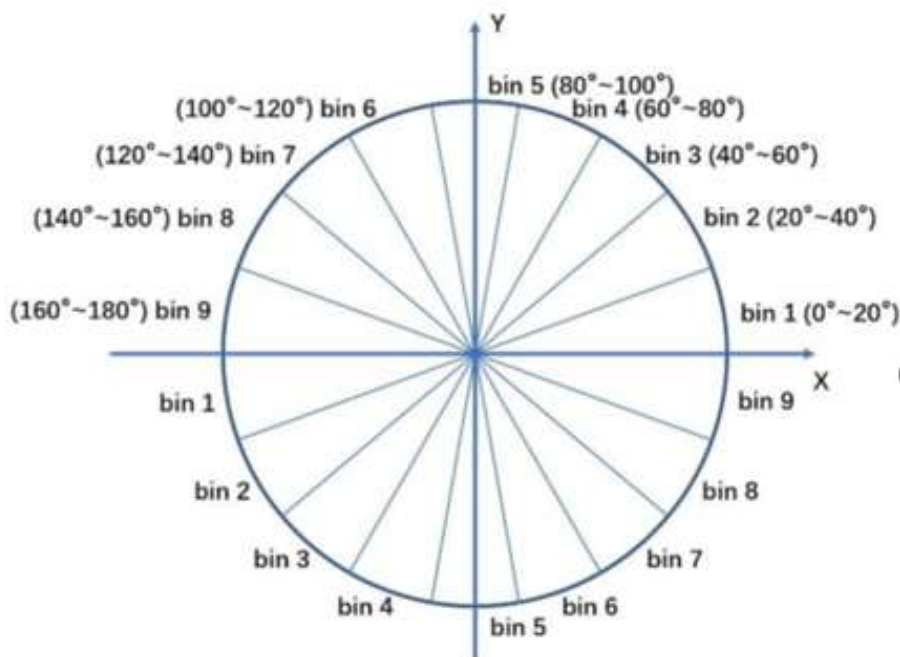
Quantize the gradient orientation into 9 bin of 20°

Magnitude = 13.6

Orientation = 36°



	16		4	
20		36		40

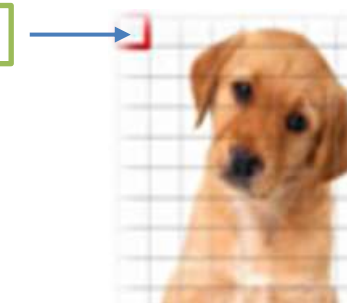


Magnitude		$4/20 \cdot 13.6$	$16/20 \cdot 13.6$						
Bin	0	20	40	60	80	100	120	140	160
Features	1	2	3	4	5	6	7	8	9

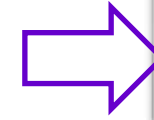
The pixel's orientation is 36° (closer to the 40-degree bin), most of its magnitude goes to the 40-degree bin.

The rest to the 20-degree bin

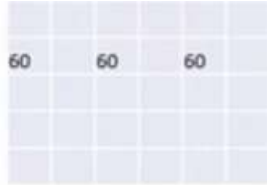
Matrix (1 x 9)



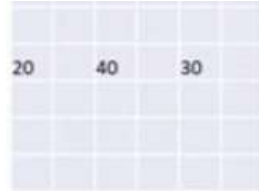
Magnitude Distribution in lecture



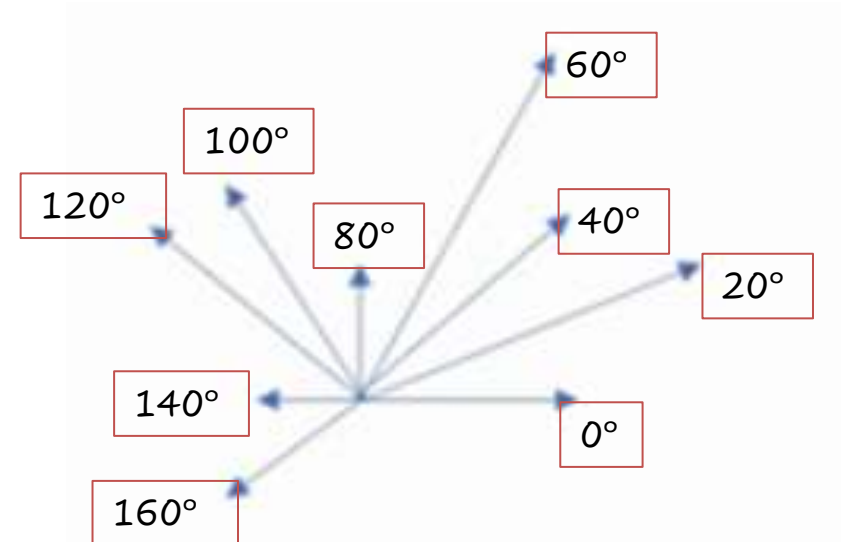
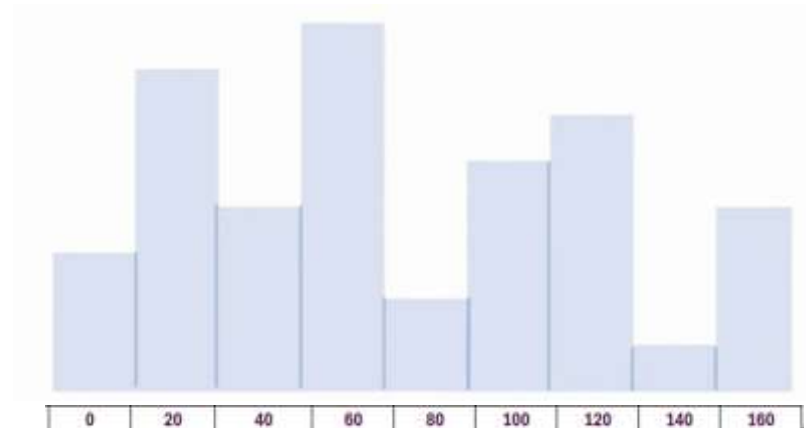
Build
histogram



A matrix block of
gradient magnitude



A matrix block of
gradient orientation



Nine feature vectors

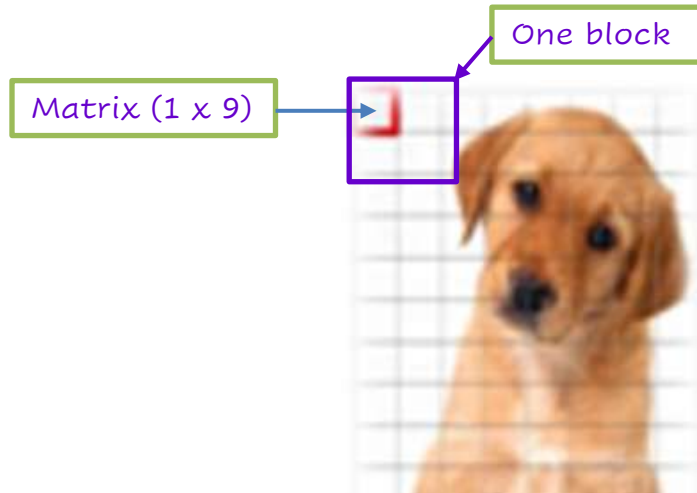
The final orientation histogram is generated
by adding all the magnitude in the every bin

Normalize across a block

Each block = concatenate the feature vectors of its four cells to form a 1x36 block feature vector.



Compute the normalization factor k as the root sum of squares of all 36 block features.



$$F = [f_1, f_2, f_3 \dots f_{36}] \quad \text{A block having 36 features}$$

$$N_s = \sqrt{f_1^2 + f_2^2 + \dots + f_{36}^2} \quad \text{Normalized scaling factor}$$

$$N_v^1 = \frac{F}{N_s} \quad \text{a block normalized vector}$$

Normalize across blocks

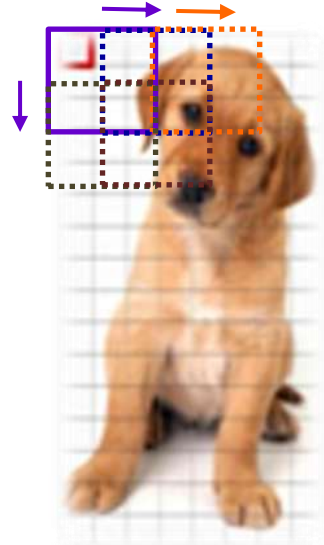
Repeat this process for all blocks (sliding across the image).



Concatenate the normalized vectors from all blocks horizontally to form the final image descriptor

Image feature descriptor (IMF) = vector (1, 3780)

This high-dimensional descriptor represents the image's edge and texture structure for use in machine learning tasks.



Due to the large number of features, dimensionality reduction techniques like Principal Component Analysis (PCA) are often used to reduce computational complexity

Why HoG is effective ?

Captures Edge/Shape Information:



Focuses on the **structure of objects**, which is **robust to lighting and color changes**.

Compact Representation:

Reduces the image to a **feature vector**, making it efficient for **machine learning algorithms like SVM**.

Scale & Illumination Invariance:

Normalization across blocks helps **handle variations in lighting and contrast**.

Applications of HoG

Human detection:

Pedestrian detection (self-driving cars and surveillance)



Vehicle detection:

Detection of cars, truck, and other vehicles in traffic analysis.

Face detection:

Pre-processing step in face recognition systems.

Object recognition:

Object detection tasks in robotics and automation

Medical imaging:

Identifying structures in X-rays, MRI and so on

Gesture recognition:

Recognizing hand or body gestures in interactive systems.

Corner and Its Importances:

Corner Definition:

Corners represent points where edges intersect, providing valuable information about the shape and structure of objects in an image.

Object Recognition:

By detecting and matching corners, it's possible to identify objects even with slight variations in viewpoint or lighting.

Image Registration:

Corner detection helps to align multiple images of the same scene by finding corresponding points.

Motion Tracking:

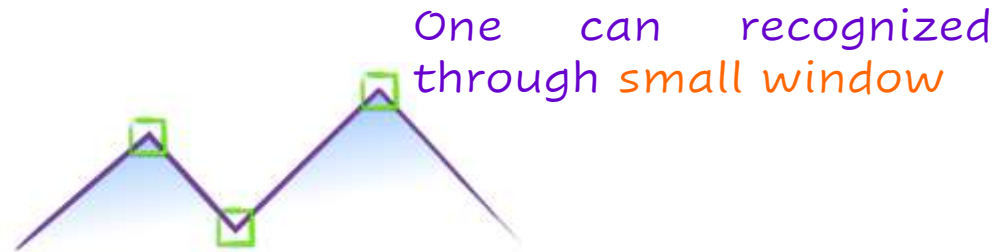
Corners can be used to track the movement of objects over time, even with partial occlusions or changes in appearance.

Applications:

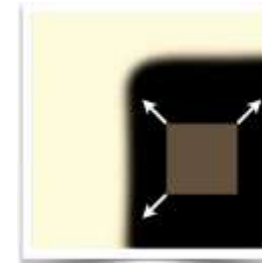
- image stitching
- 3D reconstruction
- Stereo vision

Image registration is the process of aligning different images to a common coordinate system, allowing for comparison, fusion, or analysis of data from different sources, times, or modalities

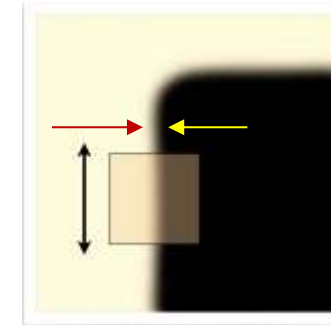
Flat, Edge, and Corner in Image:



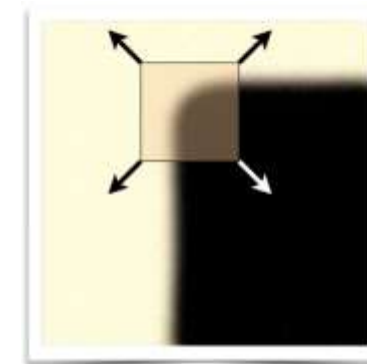
Shifting the window should give large change in intensity



"flat" region:
no change in
all directions



"edge" : no change
along the edge
direction

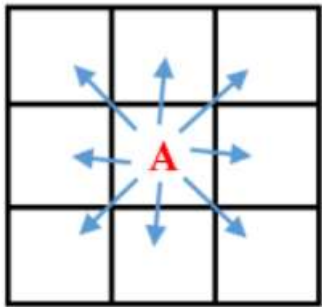


"corner" : significant
change in all directions
with small shift

Moravec Corner Detector:

The Moravec Corner Detector is one of the earliest techniques developed to detect corners in an image.

This technique compares the sum of squared differences between a patch of pixels and its shifted versions in multiple directions.



“Sum of squares of differences of pixel adjacent in all direction over each window are calculated. The minimum windows measure is selected. The interested pixels has a local maxima.

Pixel direction in 3x3 window

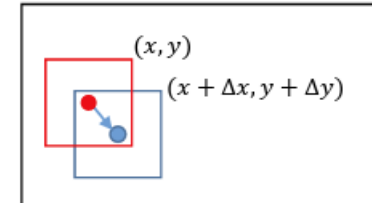
Moravec Corner Detector:

“Sums of squares of differences of pixels adjacent in each of four directions (horizontal, vertical and two diagonals) over each window are calculated, and the window's interest measure is the minimum of these four sums. Features are chosen where the interest measure has local maxima.”
[Moravec'80, PhD thesis, Chapter 5, link]

Algorithm Steps:

1. Compute SSD (Sum of Squared Differences):

For each pixel, compute the SSD for shifts in the four principal directions.



2. Find Minimum SSD:

Determine the minimum SSD value for each pixel.

$$SSD(\Delta x, \Delta y) = \sum_{x, y \in \Omega} (I(x, y) - I(x + \Delta x, y + \Delta y))^2$$

3. Thresholding:

Apply a threshold to identify potential corners.

4. Non-Maximum Suppression:

Perform non-maximum suppression to retain only the strongest corners.

Moravec detector: Example

6	3	3	6	12
3	6	6	6	6
6	15	14	0	0
7	15	0	0	0
0	7	3	1	0

6	6	6
15	14	0
15	0	0

6	3	3	6	12
3	6	6	6	6
6	15	14	0	0
7	15	0	0	0
0	7	3	1	0

$SSD1 = (3-6)^2 + (3-6)^2 + (6-6)^2 + (6-15)^2 + (6-14)^2 + (6-0)^2 + (15-15)^2 + (14-0)^2 + (0-0)^2 = 365$

6	3	3	6	12
3	6	6	6	6
6	15	14	0	0
7	15	0	0	0
0	7	3	1	0

$SSD2 = 451$

6	3	3	6	12
3	6	6	6	6
6	15	14	0	0
7	15	0	0	0
0	7	3	1	0

$SSD3 = 422$

6	3	3	6	12
3	6	6	6	6
6	15	14	0	0
7	15	0	0	0
0	7	3	1	0

$SSD4 = 576$

6	3	3	6	12
3	6	6	6	6
6	15	14	0	0
7	15	0	0	0
0	7	3	1	0

$SSD5 = 227$

6	3	3	6	12
3	6	6	6	6
6	15	14	0	0
7	15	0	0	0
0	7	3	1	0

$SSD6 = 764$

6	3	3	6	12
3	6	6	6	6
6	15	14	0	0
7	15	0	0	0
0	7	3	1	0

$SSD7 = 702$

6	3	3	6	12
3	6	6	6	6
6	15	14	0	0
7	15	0	0	0
0	7	3	1	0

$SSD8 = 702$

Threshold = 220, 227 > 220: Corner

Disadvantages of Moravec Methods:

- ❑ Strong response to edge points (can be sensitive to noise)
- ❑ Finding a good size for moving window is experimental.
- ❑ Moravec corner detects only concerns eight principles directions which will have a poor repeatability rate.

$$\text{Repeatability} = \frac{\# \text{ correspondences detected}}{\# \text{ correspondences present}}$$

Finding corners (a.k.a PCA): Harris Corner

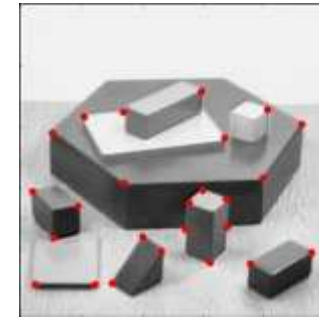
Step One: Compute image gradients over small region

Step Two: Subtract mean from each image gradient

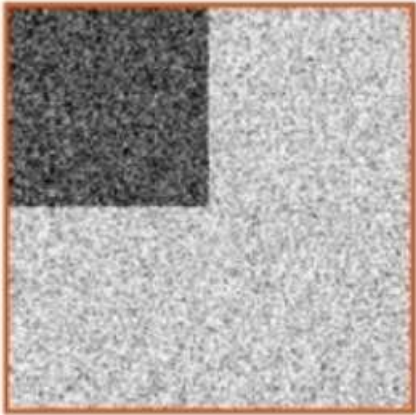
Step Three: Compute the covariance matrix

Step Four: Compute eigenvectors and eigenvalues

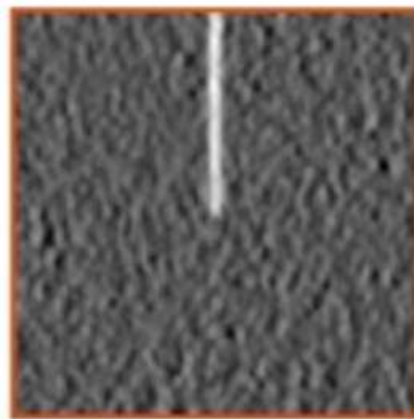
Step Five: Use threshold on eigenvalues to detect corners



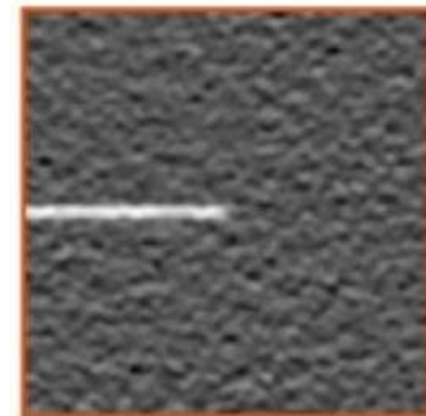
Derivatives in x- and y- directions:



image

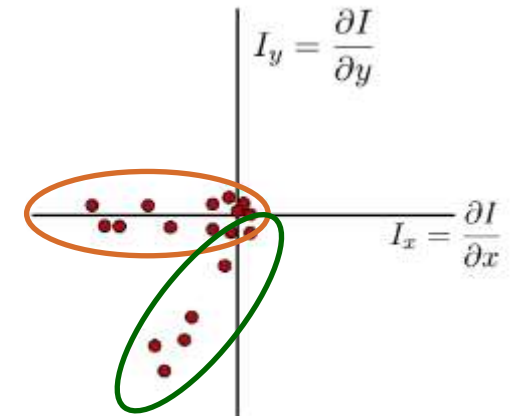
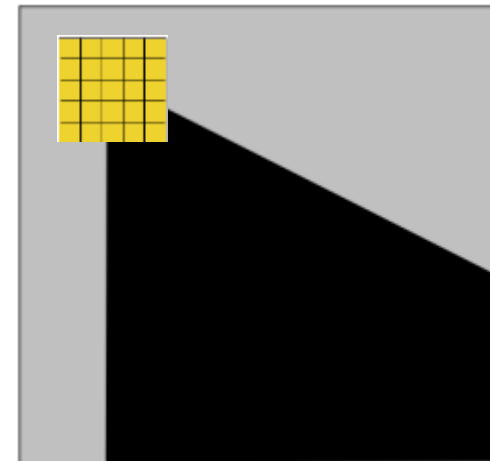
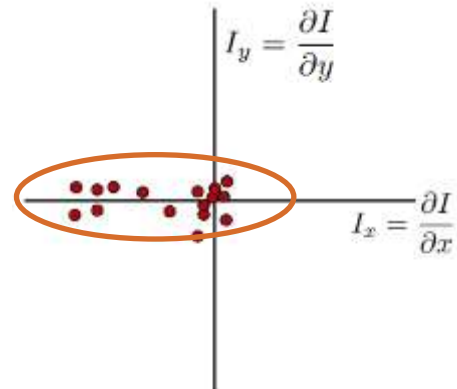
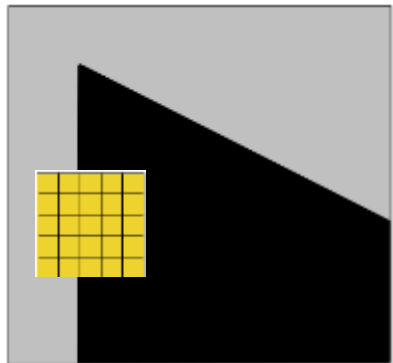
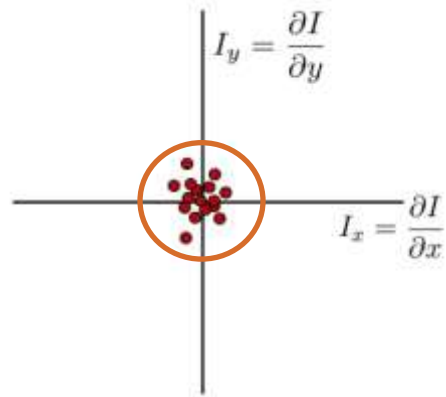
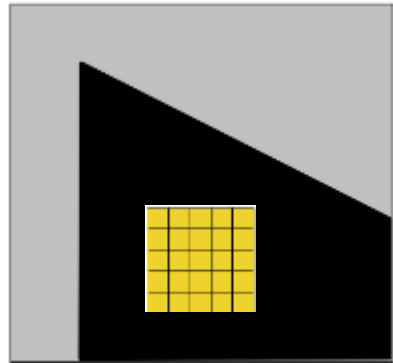


$$I_x = \frac{\partial I}{\partial x}$$



$$I_y = \frac{\partial I}{\partial y}$$

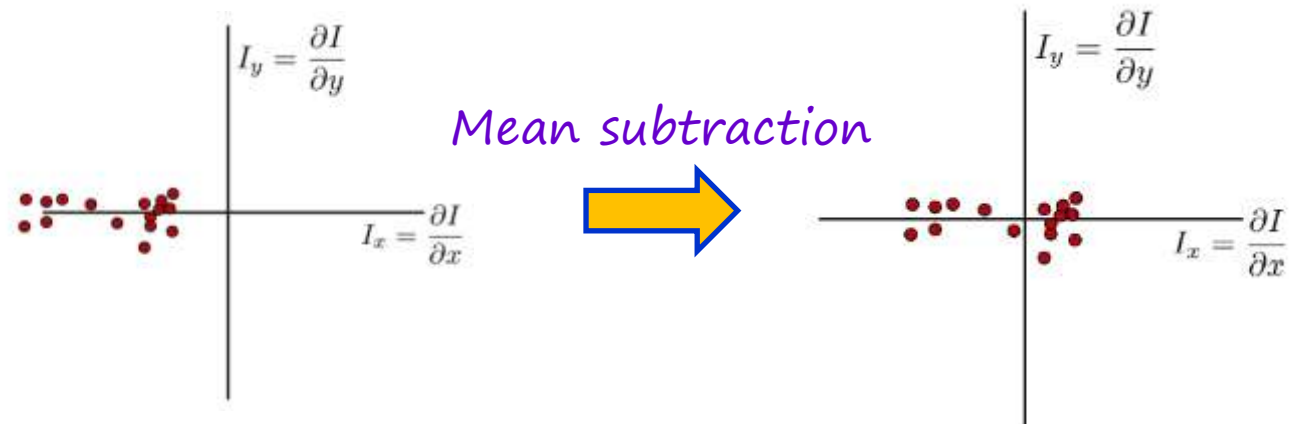
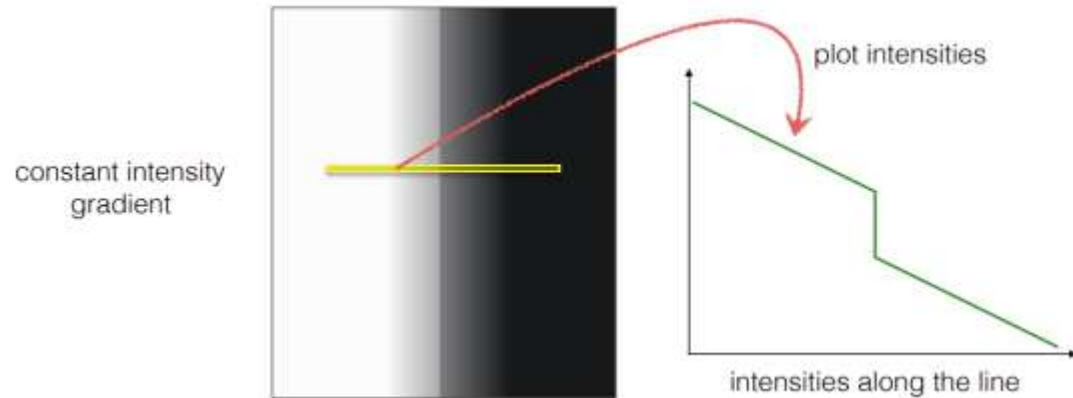
Gradient Region Distributions:



Distribution reveals edge orientation and magnitude

How do you quantify magnitude and orientation?:

Subtract the mean from each image gradient



Centered Data ('DC' offset is removed)

Error Model:

Change of intensity for the shift $[u, v]$

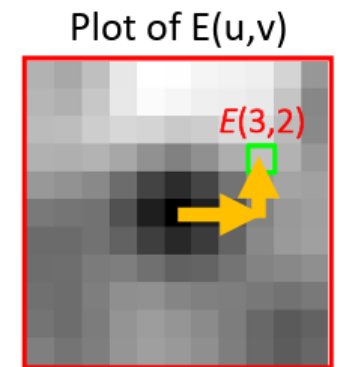
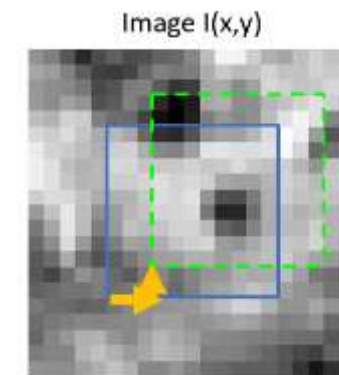
$$E(u, v) = \sum_{x, y} w(x, y) [I(x+u, y+v) - I(x, y)]^2$$

Error
function

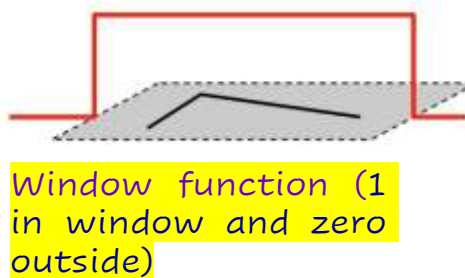
Window
function

Shifted
Intensity

intensity



$w(x, y) =$



Window function is either a rectangular window or a gaussian window which gives weights to pixels at (x, y)

Compute eigenvalues and eigenvectors:

Eigen value
↓

$$M\mathbf{e} = \lambda\mathbf{e} \quad (M - \lambda I)\mathbf{e} = 0$$

↖ ↗
Eigen vector

Eigenvalues: **Magnitude**

Larger eigenvalues signify a greater change in image intensity, indicating a significant feature (corner/edge).

Eigenvectors: **Direction:**

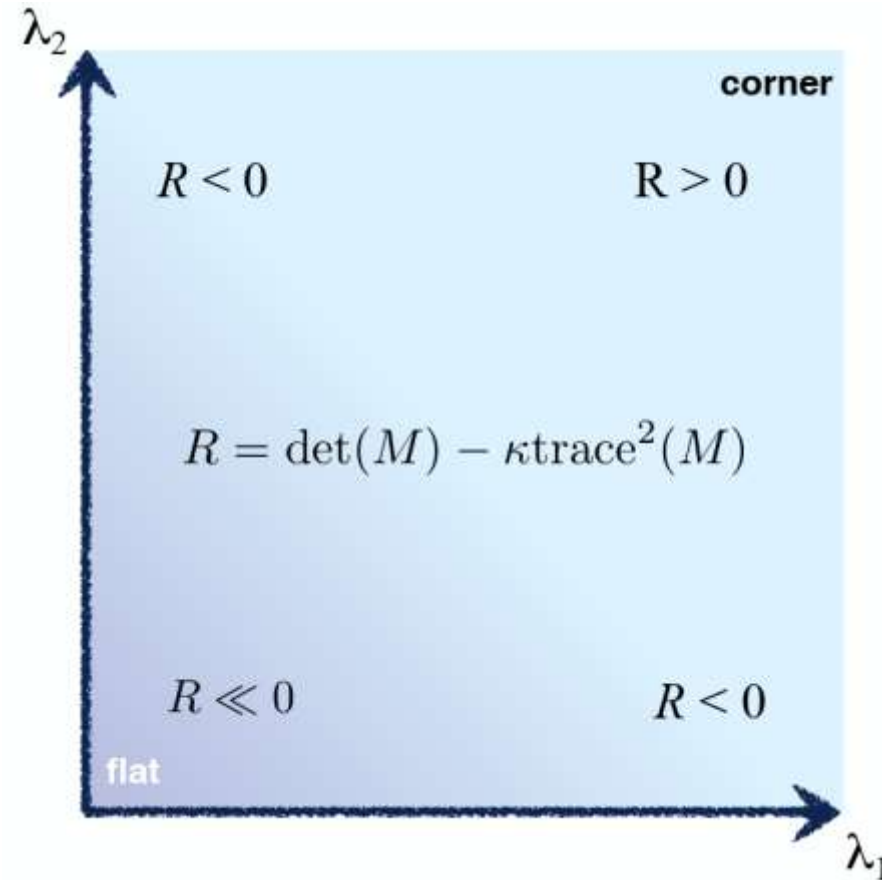
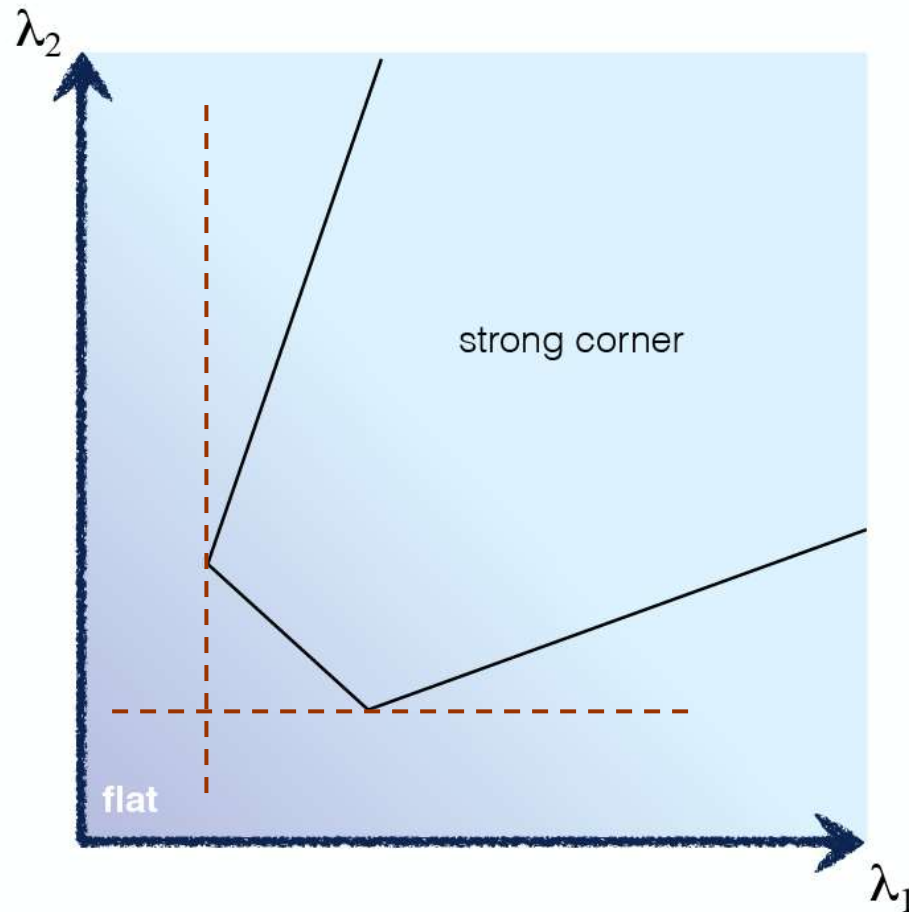
The eigenvectors point in the principal directions of intensity change (gradients).

1. Compute the determinant of $M - \lambda I$
(returns a polynomial)

2. Find the roots of polynomial $\det(M - \lambda I) = 0$
(returns eigenvalues)

3. For each eigenvalue, solve $(M - \lambda I)\mathbf{e} = 0$
(returns eigenvectors)

Use threshold on eigenvalues to detect corners



Calculation of R in Harris detector

$$\det M = \lambda_1 \lambda_2$$

$$\text{trace } M = \lambda_1 + \lambda_2$$

$$\det \begin{pmatrix} a & b \\ c & d \end{pmatrix} = ad - bc$$

$$\text{trace} \begin{pmatrix} a & b \\ c & d \end{pmatrix} = a + d$$

$$R = \det(M) - \kappa \text{trace}^2(M)$$

κ is a magic number in the range (0.04 to 0.15)

If R is larger : Corner

If R is negative : Edge

Algorithm: Harris corner detection

Step 1: Compute x and y derivatives of image

$$I_x = G_x^x * I$$

$$I_y = G_y^y * I$$

$$\begin{bmatrix} -1 & 0 & 1 \end{bmatrix}$$

$$\begin{bmatrix} 1 \\ 0 \\ -1 \end{bmatrix}$$

Step 2: Compute products of derivatives at every pixel

$$I_{x^2} = I_x \cdot I_x$$

$$I_{y^2} = I_y \cdot I_y$$

$$I_{xy} = I_x \cdot I_y$$

Step 3: Compute the sums of the products of derivatives at each pixel

$$S_{x^2} = G_{x^2} * I_{x^2}$$

$$S_{y^2} = G_{y^2} * I_{y^2}$$

$$S_{xy} = G_{xy} * I_{xy}$$

Step 4: Define the matrix at each pixel

$$M(x, y) = \begin{bmatrix} S_{x^2}(x, y) & S_{xy}(x, y) \\ S_{xy}(x, y) & S_{y^2}(x, y) \end{bmatrix}$$

Step 5: Compute the response of the detector at each pixel

$$R = \det M - k(\text{trace} M)^2$$

Step 6: Threshold on value of R; compute non-max suppression.

Harris detector: Example

0	0	1	4	9
1	0	5	7	11
1	4	9	12	16
3	8	11	14	16
3	8	11	14	16
8	10	15	16	20

-1	0	1
----	---	---

d/dx

-1
0
1

d/dy

0	0	1	4	9
-1	0	1	7	11
1	4	9	12	16
3	8	11	14	16
3	8	11	14	16
8	10	15	16	20

0	-1	1	4	9
1	0	5	7	11
1	1	9	12	16
3	8	11	14	16
3	8	11	14	16
8	10	15	16	20

0	0	1	4	9
1	4	7	6	11
1	8	8	7	16
3	8	6	5	16
3	8	11	14	16
8	10	15	16	20

I_x

0	0	1	4	9
1	4	8	8	11
1	8	8	7	16
3	6	6	4	16
3	8	11	14	16
8	10	15	16	20

I_y

$$I_x^2 = 4^2 + 7^2 + 6^2 + \dots + 5^2 = 403$$

$$I_y^2 = 4^2 + 8^2 + 8^2 + \dots + 4^2 = 381$$

$$I_x \cdot I_y = 4 \cdot 4 + 7 \cdot 8 + 6 \cdot 8 + \dots + 5 \cdot 4 = 385$$

Harris detector: Example

$$I_x^2 = 4^2 + 7^2 + 6^2 + \dots + 5^2 = 403$$

$$I_y^2 = 4^2 + 8^2 + 8^2 + \dots + 4^2 = 381$$

$$I_x \cdot I_y = 4 \cdot 4 + 7 \cdot 8 + 6 \cdot 8 + \dots + 5 \cdot 4 = 385$$

$$M = \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix}$$

$$R = \det(M) - \kappa \text{trace}^2(M)$$

k is a magic number in the range (0.04 to 0.15)

$$R = 5318 - 0.04 * (784)^2$$

$$R = -19268.24$$

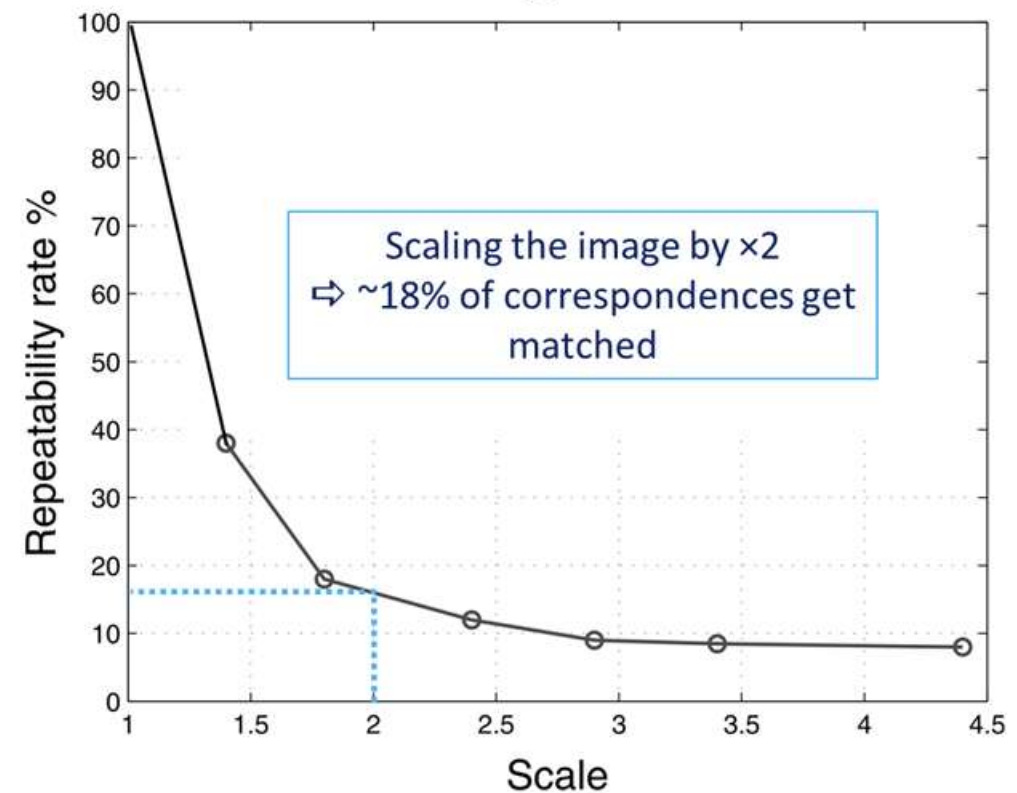
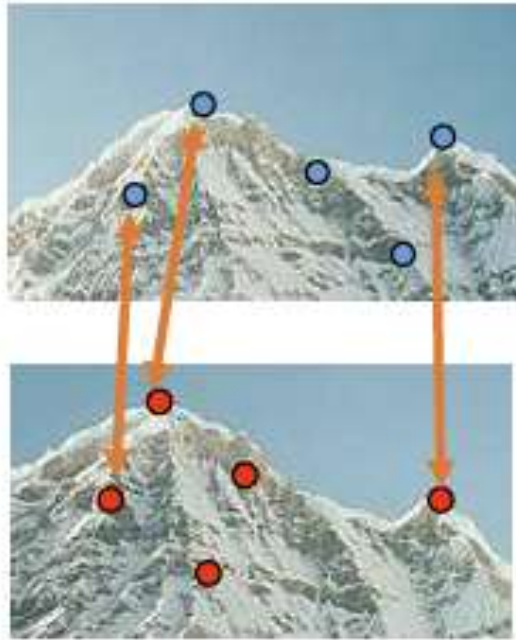
If R is larger : Corner

If R is negative : Edge

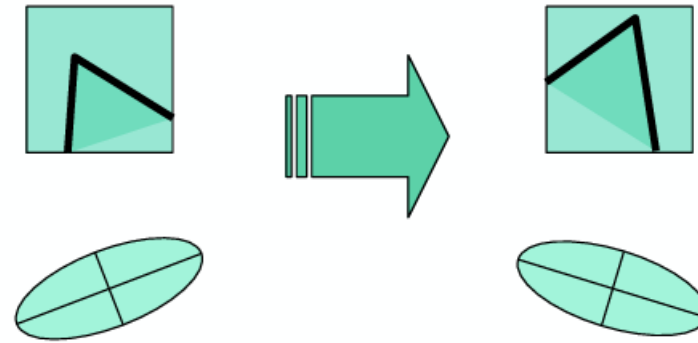
Harris detector: Properties

Repeatability of the Harris detector for different scale changes

$$\text{Repeatability} = \frac{\# \text{ correspondences detected}}{\# \text{ correspondences present}}$$



Harris corner response is rotation invariant

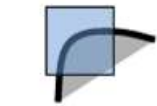


Ellipse rotates but its shape (eigenvalues) remains the same

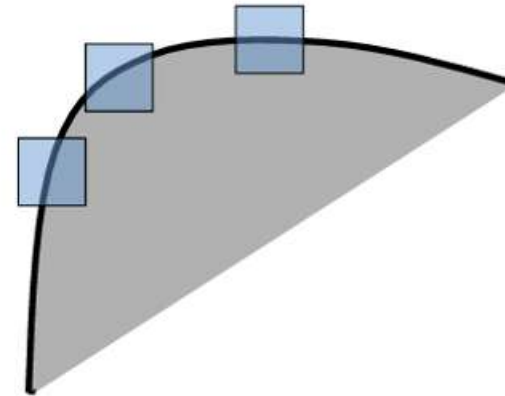
Corner response R is invariant to image rotation

Harris detector: Properties

The Harris detector is not scale invariant



Corner!



All points will be classified as edges

Shi-Tomasi detector

A corner can then be identified by checking whether the minimum of the two eigenvalues of M is larger than a certain user-defined threshold

$$R = \min(\lambda_1, \lambda_2) > \text{threshold}$$

R is called "corneriness function"

The corner detector using this criterion is called Shi-Tomasi detector

Harris vs Shi Tomasi corner response

